UNIVERSITÉ DE YAOUNDÉ I
****
ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ
****
DÉPARTEMENT DE GÉNIE INFORMATIQUE

UNIVERSITY OF YAOUNDE I
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING OF YAOUNDE
****
DEPARTMENT OF COMPUTER ENGINEERING

Optimisation du traitement des queues de m-vecteurs dans les noyaux AVX2 de MAYO-C : l'approche A2

Mémoire de Master II

Présenté et soutenu par :

Marcel Eric Angoah

En vue de l'obtention du :

Master II en Génie Informatique

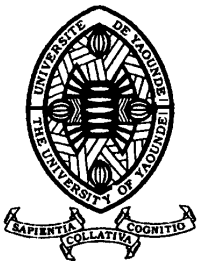
Sous la supervision de :

Dr Hervé Talé Kalachi

Année académique 2025-2026

Version provisoire (premier jet complet) — 27 octobre 2026





Optimisation du traitement des queues de m-vecteurs dans les noyaux AVX2 de MAYO-C : l'approche A2

Mémoire de Master II

Présenté et soutenu par :

Marcel Eric Angoah

En vue de l'obtention du :

Master II en Génie Informatique

Année académique 2025-2026

Soutenu le [date]

Dédicace

(*à compléter*)

Remerciements

« Une seule main ne peut nouer un fagot de bois. »
— Adage africain

Fort de cet adage, je prends la plume pour remercier celles et ceux dont le soutien m'a permis de mener à terme la rédaction de ce mémoire. Ce travail n'aurait pas été possible sans l'accompagnement de plusieurs personnes, à qui j'exprime ma sincère gratitude :

- **Dr Hervé Talé Kalachi**, mon encadrant, pour ses relectures exigeantes et les consignes méthodologiques qui ont structuré ce travail ;
- (*à compléter : membres du jury, enseignants du Département de Génie Informatique, collègues, famille, amis.*)

Abstract

MAYO is an Oil-and-Vinegar multivariate signature scheme submitted to post-quantum standardisation. In the official AVX2 C implementation, products of the public matrix P_1 by the vectors of V are computed with `vpshufb` lookups on packed $\mathbb{F}_{16}$ vectors. When the vector length m is not a multiple of 64, the official kernel covers the tail with an overlapping 32-byte load that duplicates work. We study a local change, called A2, which handles the 8-byte tail of each P_1 entry in a single 128-bit register and one xmm `vpshufb` on the low lane of the multiplication table, reducing the number of shuffles per scalar pair from 4 to 3 (MAYO-1) and from 6 to 5 (MAYO-5). We formalise the counting argument and its limits, establish the properties on which correctness rests, and measure the effect on a physical laptop with a paired protocol. Kernel gains are 12–29 %; the full-signature gain is 10.3 % for MAYO-1 but only 1.3 % for MAYO-5. Static analysis shows that in the signature context GCC 16.2.0 emits extra 32-byte stores in several loops, asymmetrically between the two versions, consistently with the measured loss; causality is not established. GCC's dumps show that the extra stores are per-iteration stores of the accumulator arrays that are not moved out of the loop; SRA is ruled out, and why the compiler makes this choice remains open.

Résumé

MAYO est un schéma de signature numérique « multivarié » de type Oil-and-Vinegar. Il est candidat à la standardisation de la cryptographie post-quantique. Dans le code C officiel (MAYO-C) pour AVX2, le programme multiplie la matrice publique P_1 par les vecteurs de V avec l'instruction `vpshufb`. Les vecteurs de $\mathbb{F}_{16}$ y sont rangés à deux éléments par octet. Quand la longueur m n'est pas un multiple de 64, le code officiel traite la fin du vecteur en relisant 32 octets déjà traités. Cela fait des calculs en double : 48 éléments sur 128 pour MAYO-1 ($m = 80$) et 50 sur 192 pour MAYO-5 ($m = 142$).

Ce mémoire étudie une petite modification, appelée A2. Elle traite la fin de 8 octets de chaque entrée de P_1 dans un seul registre de 128 bits : un seul `vpshufb` xmm remplace deux `vpshufb` ymm. Le nombre de `vpshufb` par paire de nombres passe de 4 à 3 (MAYO-1) et de 6 à 5 (MAYO-5). Nous démontrons cette limite de comptage, nous établissons les propriétés qui rendent A2 correct, puis nous mesurons l'effet sur un ordinateur portable (Intel Core i9-9880H, GCC 16.2.0). Les mesures comparent les deux versions dans les mêmes conditions, et l'unité statistique est le processus (une exécution du programme).

Voici les résultats établis. Sur les noyaux pris seuls, A2 réduit le temps de 19 à 22 % (`P1_times_Vt`) et de 12 à 29 % (`P1t_times_Vt`). Sur la signature complète, le gain est de 10,3 % pour MAYO-1, mais de seulement 1,3 % pour MAYO-5. Pour MAYO-5, il monte à 7,3 % quand toutes les sondes de mesure sont actives. En lisant le code produit par GCC 16.2.0 (sans l'exécuter), nous voyons que, sans toutes les sondes, le compilateur ajoute dans plusieurs boucles une écriture mémoire de 32 octets par accumulateur, et que ce n'est pas la même chose dans les deux versions. Les temps mesurés varient de la même façon, mais le lien de cause à effet n'est pas démontré, et un modèle à un seul paramètre exagère le gain d'environ 40 %. Ces écritures en plus sont les stockages des tableaux d'accumulateurs, refaits à chaque tour de boucle au lieu d'être faits une seule fois après la boucle. Nous écartons la passe SRA du compilateur comme cause. Pourquoi le compilateur fait ce choix reste une question ouverte.

Mots-clés : MAYO, signatures multivariées, cryptographie post-quantique, AVX2, `vpshufb`, $\mathbb{F}_{16}$, optimisation, mesure de performance.

Liste des abréviations

AVX2 : *Advanced Vector Extensions 2* (instructions SIMD 256 bits)

CT : *Constant Time* (temps constant)

DSE : *Dead Store Elimination* (passe de GCC)

FRE : *Full Redundancy Elimination* (passe de GCC)

GCC : *GNU Compiler Collection*

GFNI : *Galois Field New Instructions*

IC95 : intervalle de confiance à 95 %

KAT : *Known-Answer Test*

LIM : *Loop Invariant Motion* (passe de GCC)

MAYO : schéma de signature multivariée de type Oil-and-Vinegar

NIST : *National Institute of Standards and Technology*

OV, UOV : (*Unbalanced*) *Oil and Vinegar*

PRE : *Partial Redundancy Elimination* (passe de GCC)

SIMD : *Single Instruction, Multiple Data*

SRA : *Scalar Replacement of Aggregates* (passe de GCC)

TSC : *Time Stamp Counter*

Glossaire

- **m-vec :** vecteur de m éléments de $\mathbb{F}_{16}$, empaqueté à deux éléments par octet.
- **Limbe :** mot de 64 bits d'un accumulateur.
- **Voie (*lane*) :** moitié de 128 bits d'un registre ymm.
- **Table de multiplication (multab) :** table de 16 octets donnant, pour une paire de scalaires, les deux produits par un quartet.
- **Queue :** fin d'un m-vec qui ne remplit pas un registre complet de 32 octets.
- **Recouvrement :** chargement reculé du dernier registre, qui retraite des octets déjà traités.
- **SAFE :** variante d'A2 qui ne lit rien au-delà de l'entrée de P_1.
- **Tick TSC :** unité du compteur d'horodatage (`rdtscp`) ; ce n'est pas un cycle cœur.
- **Processus :** unité statistique : une exécution du banc, 3 001 paires.
- **Doublon :** écritures de 32 octets supplémentaires par accumulateur dans le code généré, selon la construction.
- **E1, E1b :** analyses statiques du code généré (E1b : origine du doublon, exécutée le 25 octobre 2026 sous GCC 16.2.0).
- **Fait, inférence, hypothèse :** statuts des énoncés, notés **[F]**, **[I]**, **[H]**.

Table des matières

Dédicace i

Remerciements ii

Abstract iii

Résumé iv

Liste des abréviations v

Glossaire vi

Liste des figures xi

Liste des tableaux xii

Introduction générale 1

Contexte et motivation . . . 1
Problématique . . . 2
Objectifs de l'étude . . . 2
Méthode de recherche . . . 3
Intérêt de l'étude . . . 3
Plan du mémoire . . . 3

1 Fondements et revue de littérature 6

1.1 Fondements théoriques . . . 6
1.1.1 Le schéma MAYO . . . 6
1.1.1.1 Des signatures multivariées à Oil-and-Vinegar, puis à UOV . . . 6
1.1.1.2 Le schéma MAYO . . . 7
1.1.1.3 Évaluation par produits de matrices . . . 7
1.1.1.4 Algorithmes (version simplifiée) . . . 8

1.1.2 La signature dans le code de référence . . . 8
1.1.2.1 Le déroulement d'une tentative . . . 8
1.1.2.2 Où sont les noyaux étudiés . . . 8
1.1.2.3 Rôle de P_1 et paramètres . . . 9
1.1.3 Représentation des données . . . 9
1.1.3.1 Le corps $\mathbb{F}_{16}$ et les m-vecteurs . . . 9
1.1.3.2 Empaquetage en quartets . . . 10
1.1.4 Arithmétique $\mathbb{F}_{16}$ par `vpshufb` . . . 10
1.1.4.1 Rappel : le registre AVX2 et `vpshufb` . . . 10
1.1.4.2 Principe . . . 10
1.1.4.3 Deux produits par recherche . . . 10
1.1.4.4 Recouvrement en fin de vecteur . . . 11
1.1.5 Panorama des analyses menées . . . 11
1.2 Revue de littérature . . . 12
1.2.1 Implémentations existantes . . . 12
1.2.2 Fuites par canaux auxiliaires et temps constant . . . 12
1.3 Évaluation et positionnement . . . 12

2 Méthodologie **14**
2.1 Conception d'A2 : comptage, mise en œuvre, hypothèses et limites . . . 14
2.1.1 Modèle de comptage . . . 14
2.1.2 Énoncés . . . 15
2.1.3 Conception d'A2 . . . 16
2.1.3.1 Structure du noyau . . . 16
2.1.3.2 Variante par défaut et variante SAFE (MAYO-5) . . . 16
2.1.3.3 Préconditions . . . 16
2.1.4 La limite n'implique pas l'accélération . . . 17
2.2 Propriétés de correction et de sûreté . . . 17
2.2.1 Limites de lecture de P_1 . . . 17
2.2.2 Limites d'écriture . . . 18
2.2.3 Invariant des deux voies des tables . . . 18
2.2.4 Influence des quartets de bourrage . . . 19
2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas . . . 19
2.2.6 Correction fonctionnelle . . . 20
2.3 Protocole expérimental . . . 20
2.3.1 Plateforme . . . 20

2.3.2 Code comparé . . . 21
2.3.3 Protocole de mesure apparié . . . 21
2.3.4 Unité statistique et intervalles . . . 21
2.3.5 Sessions . . . 21
2.3.6 Instrumentation de décomposition . . . 22
2.3.7 Analyse statique du code généré (E1) . . . 22
2.3.8 Protocole E1b (analyse statique de l'origine des écritures) . . . 22
2.3.9 Traçabilité et reproduction . . . 23
2.3.10 Menaces à la validité identifiées a priori . . . 23

3 Résultats et discussion **24**
3.1 Correction . . . 24
3.2 Gains sur les noyaux et sur la signature (S1, S2) . . . 24
3.3 Effet d'ordre . . . 25
3.4 Anomalie de MAYO-5 . . . 26
3.4.1 Toutes sondes actives (PH1) . . . 26
3.4.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1) 26
3.4.3 Mesure directe, sans soustraction de médianes (config. `mcalc`) . 26
3.5 Analyse statique (E1) . . . 27
3.5.1 Comptage de base . . . 27
3.5.2 Écritures supplémentaires en contexte de signature . . . 28
3.6 Instructions et temps . . . 28
3.7 Analyse statique E1b . . . 29
3.7.1 Contrôles . . . 29
3.7.2 Désactivation de passes, une à la fois . . . 29
3.7.3 Ce que montrent les sorties de GCC . . . 29
3.7.4 Interprétation autorisée . . . 30
3.8 Comparaison avec la VM . . . 31
3.9 Discussion critique . . . 31
3.9.1 Bilan : établi, déduit, supposé . . . 31
3.9.2 Pourquoi le gain se dilue . . . 31
3.9.3 Ce que la concordance statique/dynamique prouve et ne prouve pas . . . 32
3.9.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment . . . 32
3.9.5 Pistes explorées puis écartées . . . 32
3.9.6 Limites de la validation . . . 33
3.9.7 Sur l'intégration dans MAYO-C . . . 33

Conclusion générale et perspectives **35**

Références **37**

A Inventaire des archives et reproduction **38**

A.1 Versions définitives à citer 38

A.2 Identification exacte du code 39

A.3 Reproduire les analyses 39

A.4 Refaire les mesures 39

B Identification des noyaux dans le code généré (GCC 16.2.0, MAYO-5) **40**

C Exemple jouet complet de MAYO sur $\mathbb{F}_{16}$ **41**

Table des figures

1.1 Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative. 9

1.2 Chargements pour une entrée de P_1 de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40, 64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0, 71)$. 11

3.1 Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5. 25

3.2 MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat. 27

Liste des tableaux

1 Statut des contributions. 5

1.1 Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de P_1. 9

1.2 Situation d'A2 par rapport aux sources consultées. 13

2.1 Application aux paramètres actuels. 15

2.2 Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés). 17

2.3 Sessions de mesure sur machine physique (dates UTC). 22

2.4 Issues possibles d'E1b et conclusions autorisées (fixées avant exécution). 23

3.1 Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport. . . 24

3.2 Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95]. 26

3.3 Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature. 26

3.4 Décomposition directe de la signature MAYO-5 (ticks par signature). . 27

3.5 Instructions de la boucle interne par entrée de P_1 (MAYO-5) selon la construction. 28

3.6 E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques en dehors des boucles de travail, par rapport à `base`. . . 30

3.7 Ce qui est établi, déduit ou supposé. 34

B.1 Instructions / écritures de 32 octets des boucles de travail selon la construction. 40

Contexte et motivation

Une signature numérique prouve qui a écrit un message et que ce message n'a pas été modifié. Les signatures utilisées aujourd'hui reposent sur deux problèmes mathématiques difficiles : la factorisation des grands nombres et le logarithme discret. En 1994, Shor [1] a montré qu'un grand ordinateur quantique résoudrait ces deux problèmes rapidement. Ces signatures deviendraient alors vulnérables. C'est pourquoi le NIST (l'institut américain des normes) a publié en 2024 les premières normes de cryptographie « post-quantique » (FIPS 203, 204 et 205 [8]), et continue de chercher de nouvelles signatures [9]. Plusieurs candidats sont des schémas dits « multivariés ».

Parmi eux, MAYO [6] est une signature de type Oil-and-Vinegar (OV) [2, 3, 4]. Son intérêt est que ses clés et ses signatures sont petites. Pour cela, MAYO utilise un petit espace secret (l'« huile »), puis le « fouette » (*whipping* en anglais) pour obtenir un système plus grand. Le calcul d'une signature consiste surtout à faire des opérations sur le corps $\mathbb{F}_{16}$, c'est-à-dire sur des nombres de 4 bits.

Certains schémas multivariés ont déjà été cassés [5]. Il faut donc rester prudent quand on parle de leur sécurité. Ce mémoire ne traite pas de la sécurité mathématique de MAYO. Il traite de la *vitesse du programme* et de ses *qualités* (résultat correct, accès à la mémoire, temps d'exécution indépendant du secret). Il étudie une petite modification du code C officiel.

Déroulement du travail et recentrage. Le travail a commencé par une implémentation écrite par nous-même (arithmétique $\mathbb{F}_{16}$ par tables, étude d'une fuite d'information par le cache, élimination de Gauss–Jordan à temps constant). Cette étape était motivée par les attaques qui mesurent le temps d'exécution [10, 11]. Sur demande de l'encadrant, nous sommes ensuite passés au code C *officiel* (MAYO-C), afin d'obtenir des gains comparables à ce qui existe déjà. Ce mémoire ne porte que sur cette seconde phase. La première n'est pas reprise, sauf quand un résultat négatif éclaire une décision (section 3.9).

Problématique

Dans le code officiel pour AVX2, un m-vecteur de $\mathbb{F}_{16}$ (une suite de m éléments) est rangé à deux éléments par octet, soit $m/2$ octets. Pour le multiplier par un nombre, le code utilise l'instruction `vpshufb`. Cette instruction fait 32 recherches dans une table de 16 octets, et le code l'utilise pour obtenir, en une fois, les produits par *deux* nombres. Un registre de 256 bits traite donc 64 éléments à la fois. Quand m n'est pas un multiple de 64, il reste un petit morceau à la fin du vecteur. Le code officiel le traite en relisant les 32 derniers octets, ce qui *recouvre* (relit) une partie déjà traitée. L'article d'optimisation [7] décrit ce recouvrement pour $m = 96$ (MAYO-3) : il fait des calculs en double.

Pour les paramètres étudiés ici, le calcul en double est important : 48 éléments refaits sur 128 pour MAYO-1 ($m = 80$, soit 37,5 %) et 50 sur 192 pour MAYO-5 ($m = 142$, soit 26 %). La question centrale est :

Question centrale. Peut-on réduire ce calcul en double en traitant la fin du vecteur de façon spéciale ? Quelle est la limite exacte, en nombre d'instructions `vpshufb` ? Et ce gain se retrouve-t-il, de façon sûre et démontrable, sur le temps de la signature complète ?

Cette question sépare trois choses que l'on confond souvent : (i) une *limite de comptage* (un résultat mathématique, dans un modèle précis), (ii) sa *mise en œuvre* par un algorithme (à montrer et à tester) et (iii) l'*effet sur le temps* (un résultat d'expérience, qui dépend du compilateur et de la machine).

Objectifs de l'étude

QR1 Quel est le plus petit nombre d'instructions `vpshufb` par paire de nombres pour un m-vecteur, avec des tables de 16 octets ? Pour quelles valeurs de m le code officiel est-il déjà optimal ?

QR2 Existe-t-il une mise en œuvre sûre de cette limite pour MAYO-1 et MAYO-5, c'est-à-dire sans lecture en dehors des données et sans adresse qui dépende du secret ?

QR3 Quel gain mesure-t-on sur les noyaux `P1_times_Vt` et `P1t_times_Vt`, puis sur `mayo_sign`, sur une machine physique, avec un protocole dont les limites sont chiffrées ?

QR4 Si le gain dans la signature complète est différent du gain sur le noyau seul, que montre l'analyse du code produit par le compilateur, et que reste-t-il à démontrer ?

Les objectifs en découlent : **O1** formaliser le comptage (limite, mise en œuvre, portée) ; **O2** décrire la conception d'A2 et les propriétés dont dépend sa correction ; **O3** mesurer avec un protocole que l'on peut refaire ; **O4** expliquer l'écart entre le gain sur le noyau seul et le gain dans la signature ; **O5** situer le travail par rapport aux implémentations existantes, sans affirmer plus que ce que les sources consultées permettent.

Nous séparons ce qui est établi de ce qui reste ouvert. Le tableau 1 donne le statut de chaque contribution.

Méthode de recherche

La démarche combine le raisonnement et l'expérience. Le comptage est démontré dans un modèle précis. La mise en œuvre est justifiée par le code et testée. Les gains sont mesurés sur une machine physique, en comparant les deux versions dans les mêmes conditions ; l'unité statistique est le processus (une exécution du programme). Le code produit par le compilateur est lu et analysé sans exécution (analyse « statique ») pour interpréter les écarts. Chaque résultat porte une étiquette qui dit s'il est un fait, une déduction ou une hypothèse. Les scripts, les données brutes et les empreintes sont publiés pour que l'on puisse tout refaire.

Le travail se limite aux noyaux `P1_times_Vt` et `P1t_times_Vt` pour MAYO-1 et MAYO-5. MAYO-2 ($m = 64$) et MAYO-3 ($m = 108$) servent de témoins : pour eux, aucun gain n'est possible avec les tables de 16 octets (proposition 2.3). Nous ne proposons aucune autre optimisation. Les noyaux optimisés ne sont pas modifiés pendant l'analyse, et la décision de les intégrer dans MAYO-C est volontairement repoussée. Le fait de ne pas trouver une technique dans les sources consultées ne prouve pas qu'elle n'existe pas ailleurs : nous ne prétendons pas à une nouveauté absolue.

Les résultats sont étiquetés ainsi : **[F]** *fait* (lu dans le code produit ou dans des mesures publiées), **[I]** *déduction* tirée des faits, **[H]** *hypothèse* non testée. Pour les propriétés de sûreté de la section 2.2, **[C]** veut dire « justifiée par le code » et **[E]** « vérifiée par des tests » ; une propriété **[E]** seule n'est pas une démonstration. Les durées sont mesurées en « ticks » du compteur TSC du processeur (instruction rdtscp), et non en cycles d'horloge du cœur.

Intérêt de l'étude

Pour ceux qui font évoluer ou utilisent MAYO-C, l'étude donne une limite précise de ce que l'on peut gagner en nombre d'instructions `vpshufb`, une mise en œuvre vérifiée pour deux jeux de paramètres, et surtout un avertissement : un gain mesuré sur un noyau seul ne se retrouve pas automatiquement sur la signature complète, parce que le code produit par le compilateur change selon le contexte. Pour la méthode, le mémoire propose un protocole de mesure et d'analyse qui sépare les faits des hypothèses et que l'on peut refaire.

Plan du mémoire

Le mémoire comprend trois chapitres, entourés par cette introduction et par une conclusion. Le chapitre 1 présente MAYO, le calcul par `vpshufb` et les implémentations existantes, puis situe A2. Le chapitre 2 formalise le comptage et décrit A2 (section 2.1), établit ses propriétés de correction et de sûreté (section 2.2) et détaille le protocole

d'expérience (section 2.3). Le chapitre 3 présente les résultats et les discute en séparant faits, déductions et hypothèses. La conclusion rappelle les contributions, les limites et les suites possibles.

Tab. 1 : Statut des contributions.

	Contribution	Statut
C1	Comptage : nombre minimal d'instructions $\lceil m/32 \rceil$, schéma officiel $2\lceil m/64 \rceil$, écart $\Delta(m)$ (prop. 2.3–2.5)	établi (preuve)
C2	Réalisation de A2 (fin de vecteur $\leq$ 16 éléments) pour MAYO-1 et MAYO-5, variante SAFE qui ne lit rien en dehors de P1	établi (code, KAT, tests)
C3	Gains sur noyaux isolés (12–29 %) et sur `mayo_sign` (MAYO-1 : 10,3 % ; MAYO-5 : 1,3 %) sur une machine et avec un compilateur	établi, avec limites
C4	Localisation de la perte de gain de MAYO-5 dans `m_calculate`, quand on mesure la signature complète	établi (mesure directe)
C5	Écritures mémoire en plus, et inégales entre les deux versions, dans le code produit par GCC 16.2.0 ; elles correspondent à la perte de C4 ; l'analyse E1b les identifie à des stockages de `temp[]/tt[]` dans la boucle ; la passe SRA est écartée	constat établi sur le code ; lien avec le temps *non* démontré
C6	Dépôt public (scripts, données brutes, empreintes) qui permet de refaire les 9 analyses	établi
Q1	Pourquoi GCC laisse ces écritures dans la boucle quand toutes les sondes de mesure ne sont pas actives (quelle passe du compilateur ?)	*ouvert* (fichiers d'analyse `lim/pre` non produits)
Q2	Lien de cause à effet entre ces écritures et la perte de gain, avec un modèle qui n'exagère pas leur effet	*ouvert*
Q3	Variante pour une fin de vecteur de 17 à 32 éléments ; autres compilateurs et autres processeurs	*ouvert* (non implémenté)
Q4	Temps d'exécution indépendant du secret, démontré formellement	*non fourni*

Chapitre 1

Fondements théoriques et revue de littérature

Ce chapitre donne au lecteur ce qui est nécessaire pour suivre la suite sans autre source : le schéma de signature MAYO et ses algorithmes (section 1.1.1), leur traduction dans le code de référence et la place des noyaux étudiés (section 1.1.2), la représentation des données et l'arithmétique par `vpshufb` sur $\mathbb{F}_{16}$ (section 1.1.4), un panorama des analyses menées (section 1.1.5), puis les implémentations existantes et le positionnement de l'optimisation étudiée (sections 1.2 et suivantes).

1.1 Fondements théoriques

1.1.1 Le schéma MAYO

1.1.1.1 Des signatures multivariées à Oil-and-Vinegar, puis à UOV

Une signature « multivariée » repose sur une fonction publique $\mathcal{P} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ formée de m polynômes de degré 2 en n variables. Signer un message M, c'est trouver un point s tel que $\mathcal{P}(s) = H(M)$, où H est une fonction de hachage (avec un « sel » aléatoire) qui donne m valeurs de $\mathbb{F}_q$. Vérifier une signature, c'est simplement calculer $\mathcal{P}(s)$. Le schéma Oil-and-Vinegar (OV), proposé par Patarin en 1997 [2], cache un secret : un sous-espace $\mathcal{O} \subset \mathbb{F}_q^n$ (« l'huile »), de dimension o, sur lequel $\mathcal{P}$ vaut zéro. Les n variables se séparent en $v = n - o$ variables « vinaigre » et o variables « huile ». Dans OV, on prend $o = m$. Pour signer, on choisit les variables vinaigre au hasard. Il reste alors un *système linéaire* de m équations avec o inconnues (les variables huile), parce que, dans la base secrète, aucun terme ne multiplie deux variables huile entre elles. On le résout directement.

Dans la version d'origine, dite *équilibrée*, on avait $v = o$. Cette version a été cassée en 1998 par Kipnis et Shamir [3]. La correction, proposée en 1999 par Kipnis, Patarin et Goubin, s'appelle *Unbalanced Oil and Vinegar* (UOV) [4] : on prend $v > o$ (beaucoup plus de vinaigre que d'huile) et on garde $o = m$. La signature se déroule comme dans OV (vinaigre au hasard, puis système linéaire de m équations). Mais l'attaque contre la version équilibrée ne marche plus, et UOV n'a pas été cassé depuis. Son défaut est la

taille de la clé publique : $n = v + o$ est grand, et la clé contient m matrices triangulaires de taille $n \times n$.

1.1.1.2 Le schéma MAYO

MAYO [6, 7] part d'UOV (où $o = m$) et choisit un espace d'huile *trop petit* ($o < m$). Cela réduit beaucoup la taille des clés, mais on ne peut plus résoudre le système directement : il y a m équations pour seulement o inconnues. La solution est le « fouettage » (*whipping*). On étend la fonction publique $\mathcal{P}$ en une fonction $\mathcal{P}^\star : \mathbb{F}_q^{kn} \to \mathbb{F}_q^m$ qui prend k blocs de n variables :

$$\mathcal{P}^\star(x_1, \ldots, x_k) = \sum_{i=1}^{k} E_{ii}\, \mathcal{P}(x_i) + \sum_{i=1}^{k} \sum_{j=i+1}^{k} E_{ij}\, \mathcal{P}'(x_i, x_j), \qquad \mathcal{P}'(x, y) = \mathcal{P}(x+y) - \mathcal{P}(x) - \mathcal{P}(y),$$

où les matrices publiques $E_{ij} \in \mathbb{F}_q^{m \times m}$ servent à multiplier par des puissances de X, modulo un polynôme irréductible de degré m. Une signature est $S = (s_1, \ldots, s_k)$; elle est valide si $\mathcal{P}^\star(S) = H(M)$. Ici $q = 16$. Les paramètres m, k, o, n définissent quatre jeux (MAYO-1, 2, 3 et 5), selon le niveau de sécurité demandé par le NIST [9]. Les inconnues sont maintenant $k \cdot o$, ce qui permet d'avoir assez d'inconnues pour les m équations.

1.1.1.3 Évaluation par produits de matrices

Les m polynômes de $\mathcal{P}$ s'écrivent $p_\ell(x) = x^\top P_\ell\, x$, où P_ℓ est une matrice triangulaire supérieure. On la découpe en blocs :

$$P_\ell = \begin{pmatrix} P_\ell^{(1)} & P_\ell^{(2)} \\ 0 & P_\ell^{(3)} \end{pmatrix},$$

où $P_\ell^{(1)}$ est de taille $v \times v$ avec $v = n - o$ (notée V dans le code et dans le tableau 1.1), $P_\ell^{(2)}$ de taille $v \times o$ et $P_\ell^{(3)}$ de taille $o \times o$. Les blocs $P^{(1)}$ et $P^{(2)}$ sont recalculés à partir d'une graine par une fonction à base d'AES ; seul $P^{(3)}$ est stocké dans la clé publique [7]. Soit S la matrice dont les colonnes sont $s_1, \ldots, s_k$. Tout se déduit de $S^\top P_\ell S$: $\mathcal{P}(s_i)_\ell$ est l'élément (i, i) de cette matrice, et $\mathcal{P}'(s_i, s_j)_\ell$ est la somme des éléments (i, j) et (j, i). *Ce sont donc les produits de matrices qui coûtent le plus cher*, d'où l'intérêt de les optimiser [7].

$$P_\ell = \begin{pmatrix} P_\ell^{(1)} & P_\ell^{(2)} \\ 0 & P_\ell^{(3)} \end{pmatrix},$$

où $P_\ell^{(1)}$ est de taille $v \times v$ avec $v = n - o$ (noté V dans le code et au tableau 1.1), $P_\ell^{(2)}$ de taille $v \times o$ et $P_\ell^{(3)}$ de taille $o \times o$. Les blocs $P^{(1)}$ et $P^{(2)}$ sont dérivés d'une graine par une fonction d'expansion à base d'AES ; seul $P^{(3)}$ est stocké dans la clé publique [7]. Si S est la matrice dont les colonnes sont $s_1, \ldots, s_k$, tout se déduit de $S^\top P_\ell S$: $\mathcal{P}(s_i)_\ell$ est son élément diagonal (i, i) et $\mathcal{P}'(s_i, s_j)_\ell$ la somme des éléments (i, j) et (j, i). *Les produits matrice × matrice dominent donc le coût*, ce qui motive leur optimisation [7].

1.1.1.4 Algorithmes (version simplifiée)

KeyGen. Tirer `seed_sk` ; en dériver `seed_pk` et $O \in \mathbb{F}_{16}^{v \times o}$; développer $\{P_\ell^{(1)}, P_\ell^{(2)}\}$ depuis `seed_pk` ; calculer $P_\ell^{(3)} = \mathrm{Upper}(-O^\top (P_\ell^{(1)} O - P_\ell^{(2)}))$. Clé publique : `seed_pk` et $\{P_\ell^{(3)}\}$; clé secrète : `seed_sk`.
Sign(M). (1) Tirer un sel et calculer $t = H(M \,\|\, \text{sel}) \in \mathbb{F}_{16}^m$. (2) Tirer $V \in \mathbb{F}_{16}^{k \times v}$ (les « vinaigres »). (3) Pour chaque $\ell < m$: $L_\ell = (P_\ell^{(1)} + P_\ell^{(1)\top}) O + P_\ell^{(2)}$; $M_\ell = V\, L_\ell$; $Y_\ell = V\, P_\ell^{(1)}\, V^\top$. (4) Assembler le système linéaire $Ax = y$, où A provient des M_ℓ et $y = t +$(combinaison des Y_ℓ). (5) Si le système n'a pas de solution, retourner à l'étape 2 (nouveaux vinaigres) ; sinon tirer une solution x au hasard. (6) Signature : $S = (V + (OX)^\top, X)$, où X est x mis en matrice $k \times o$.
Verify. Redévelopper $P^{(1)}, P^{(2)}$; calculer $t = H(M \,\|\, \text{sel})$; calculer $Y_\ell = S \begin{pmatrix} P_\ell^{(1)} & P_\ell^{(2)} \\ 0 & P_\ell^{(3)} \end{pmatrix} S^\top$, combiner en $\mathcal{P}^\star(S)$; accepter si le résultat égale t.

La signature fait donc intervenir trois choses : des produits de matrices (pour M_ℓ et Y_ℓ, à partir de la clé publique développée $P^{(1)}$, $P^{(2)}$), la résolution d'un système linéaire par élimination de Gauss en temps constant, et des reprises quand le système n'a pas de solution. D'après [7], ce sont le calcul des évaluations partielles de $\mathcal{P}$ et l'élimination de Gauss qui coûtent le plus ; l'assemblage de A et de y ne coûte presque rien.

Un exemple complet (génération de clé, signature, vérification) sur $\mathbb{F}_{16}$ avec de très petits paramètres est donné à l'annexe C.

1.1.2 La signature dans le code de référence

1.1.2.1 Le déroulement d'une tentative

Dans le code de référence (annexe A), la fonction `mayo_sign_signature` calcule le sel et t, puis fait une boucle d'au plus 256 tentatives (un octet sert de compteur). Chaque tentative enchaîne les étapes suivantes (figure 1.1) :

1. tirer et décoder les vinaigres V (avec SHAKE256) ;
2. `m_calculate_M_and_VPV` : calculer les matrices M_ℓ et tous les produits $v_i P^{(1)} v_j$ (c'est ici que se trouvent les noyaux étudiés) ;
3. `compute_rhs` (second membre y) puis `compute_A` (matrice A) ;
4. `sample_solution` : résoudre $Ax = y$; en cas d'échec, nouvelle tentative ;
5. `m_compute_s` : assembler la signature $s_i = [\, v_i + Ox_i \mid x_i \,]$.

1.1.2.2 Où sont les noyaux étudiés

Dans `m_calculate_M_and_VPV` (chemin à L implicite, celui où les deux noyaux sont utilisés), les opérations se font dans cet ordre : construction des tables de multiplication de V (`mayo_V_multabs`), puis :

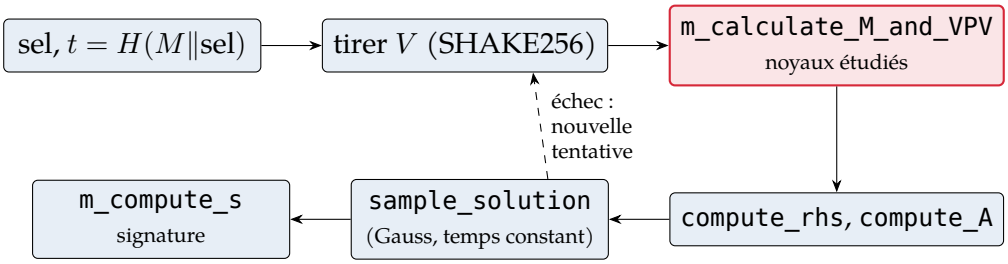


Fig. 1.1 : Une tentative de signature dans le code de référence. Les noyaux optimisés se trouvent dans l'étape en rouge, répétée à chaque tentative.

- `P1_times_Vt` : accumule $P^{(1)} \cdot V^\top$ dans un tampon ; `Vt_times_Pv` en déduit $VP^{(1)}V^\top$, c'est-à-dire tous les $v_i P^{(1)} v_j$;
- `P1t_times_Vt` : accumule la contribution transposée $P^{(1)\top}V^\top$ sur le même tampon, qui devient $(P^{(1)}+P^{(1)\top})V^\top$ (la diagonale s'annule en caractéristique 2) ; le calcul de M_ℓ réutilise ce produit, puis `Ut_times_O` et `Vt_times_L` complètent M_ℓ.

Les noyaux `P1_times_Vt` et `P1t_times_Vt` calculent donc des produits de la *partie vinaigre de la clé publique* $P^{(1)}$ par les multiples de V. Ce sont les seuls que ce mémoire optimise (variante A2), pour MAYO-1 et MAYO-5.

1.1.2.3 Rôle de P_1 et paramètres

P_1 est une matrice triangulaire supérieure dont chaque case est un m-vecteur de $\mathbb{F}_{16}^m$. Elle est rangée à plat, une entrée après l'autre : l'entrée numéro e commence à la position $e \cdot b$. Chaque noyau calcule, pour chaque entrée de P_1 et chaque vecteur de V, le produit d'un nombre de $\mathbb{F}_{16}$ par le m-vecteur de l'entrée, et l'ajoute à un accumulateur de L « limbes ». Les bornes des boucles (V, K) et les adresses de lecture ($e \cdot b$) ne dépendent que de paramètres publics. Le tableau 1.1 donne les valeurs utiles.

Tab. 1.1 : Paramètres des noyaux étudiés. m : longueur du m-vecteur ; $b = m/2$: octets par m-vecteur ; L : limbes de 64 bits d'un accumulateur ; K : nombre de vecteurs de V ; V : dimension de la partie vinaigre ; $N_{\text{ent}} = V(V+1)/2$: nombre d'entrées de P_1.

Paramètre	m	b	L	K	V	N_{ent}	$m \bmod 64$
MAYO-1	80	40	5	10	80	3 240	16
MAYO-2	64	32	4	–	73	–	0
MAYO-3	108	54	–	–	–	–	44
MAYO-5	142	71	9	12	142	10 153	14

1.1.3 Représentation des données

1.1.3.1 Le corps $\mathbb{F}_{16}$ et les m-vecteurs

$\mathbb{F}_{16}$ est construit comme $\mathbb{F}_2[x]/(x^4+x+1)$ (commentaire du code, `simple_arithmetic.h`). Un élément tient dans un *quartet* (4 bits) et l'addition est le OU exclusif. L'unité de

calcul de MAYO est le m-*vecteur* : la liste des éléments qui occupent la même case (i, j) dans les m matrices P_ℓ. On calcule ainsi sur les m composantes en parallèle, ce qui convient bien à la vectorisation [7].

1.1.3.2 Empaquetage en quartets

Un m-vecteur occupe $b = m/2$ octets : deux éléments par octet, l'élément d'indice pair dans les 4 bits de poids faible. Les accumulateurs sont découpés en *limbes* de 64 bits, soit 16 éléments par limbe. Il faut donc $L = \lceil m/16 \rceil$ limbes, avec éventuellement des quartets de bourrage (remplissage) dans le dernier limbe ($m = 142 : L = 9$, 144 positions, dont deux de bourrage). Cette représentation par quartets vient de [7], qui l'a préférée à la représentation « bitsliced » (par plans de bits) de la première version. Elle permet d'utiliser des instructions qui permutent les octets pour multiplier, au prix d'un changement du format de la clé.

1.1.4 Arithmétique $\mathbb{F}_{16}$ par `vpshufb`

1.1.4.1 Rappel : le registre AVX2 et `vpshufb`

Un registre AVX2 `ymm` contient 256 bits, soit deux *voies* de 128 bits (`xmm` désigne la moitié basse). La plupart des instructions, dont `vpshufb`, travaillent séparément sur chaque voie. `vpshufb` reçoit une table de 16 octets (une par voie) et un registre d'index. Pour chaque octet d'index, elle écrit l'élément de la table désigné par les 4 bits de poids faible de l'index, ou 0 si le bit de poids fort de l'index vaut 1. Un registre `ymm` fait donc 32 recherches avec une seule instruction.

1.1.4.2 Principe

Avec des éléments de 4 bits, multiplier un vecteur par un nombre c revient à chercher chaque élément x dans une table de 16 cases qui contient $c \cdot x$. L'instruction `vpshufb` (`_mm256_shuffle_epi8`) fait cette recherche pour chaque octet d'un registre d'index. C'est la méthode des « quatre Russes » [12] : on remplace des multiplications par des lectures dans une table. Pour calculer $A \cdot B$, on découpe A en bandes de t colonnes et B en bandes de t lignes. Pour chaque bande de B, on prépare toutes les combinaisons de ses lignes (16^t entrées), puis chaque ligne de A sert d'indice dans cette table. Ici la table tient dans un registre, donc $t = 1$ (voir aussi [13] pour $\mathbb{F}_2$). Les tables dépendent de V (qui est secret), mais elles ne servent que de données pour `vpshufb` : elles ne servent jamais d'adresse mémoire.

1.1.4.3 Deux produits par recherche

Pour une paire de nombres (c_{2p}, c_{2p+1}), la table $T_p[x] = (c_{2p}x) \oplus \big((c_{2p+1}x) \ll 4\big)$ donne *les deux* produits dans un seul octet. Le code range T_p dans un registre de 256 bits dont les deux voies sont identiques (lemme 2.7). Un registre de données de 32 octets contient 64 éléments. Il faut séparer les quartets impairs et pairs (ET avec `0x0f`, puis décalage

de 4 bits), donc deux `vpshufb` par registre et par paire. Les accumulateurs « impairs » et « pairs » restent séparés jusqu'à la fin de la ligne, où on les remet ensemble. Cette technique est décrite dans l'article *Nibbling MAYO* [7].

1.1.4.4 Recouvrement en fin de vecteur

Pour $m \leq 64$, $64 < m \leq 128$ et $128 < m \leq 192$, le code officiel utilise respectivement 1, 2 et 3 chargements de 32 octets. Le dernier est *reculé* pour se terminer à l'octet b (adresse `vec + M_VEC_LIMBS_MAX - 4` limbes). Les octets communs sont donc traités deux fois, et les écritures finales se recouvrent avec les mêmes valeurs. Pour MAYO-5, on a les chargements $[0, 32)$, $[32, 64)$ et $[40, 72)$: 192 éléments sont calculés pour 142 utiles, et le dernier chargement lit un octet de trop après l'entrée de P_1 (figure 1.2).

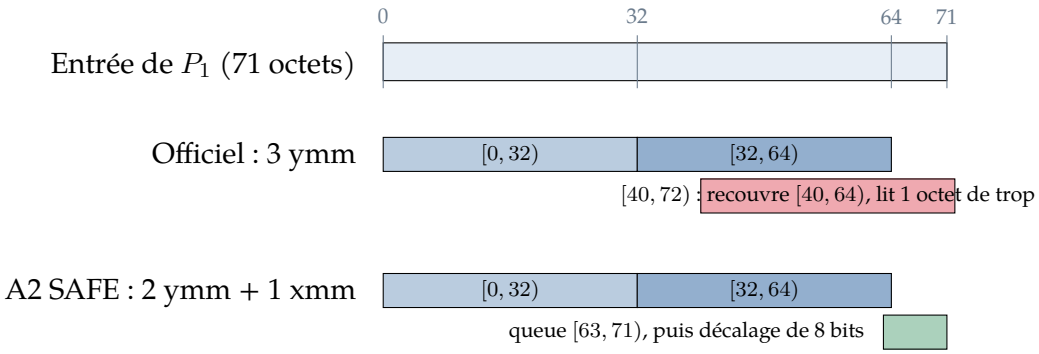


Fig. 1.2 : Chargements pour une entrée de P_1 de MAYO-5 (octets). Le noyau officiel recouvre l'intervalle $[40, 64)$ et dépasse l'entrée d'un octet ; A2 SAFE lit exactement $[0, 71)$.

1.1.5 Panorama des analyses menées

Pour s'y retrouver dans la suite, voici les analyses et ce qu'elles établissent (détails aux chapitres 2 et 3) :

- **Comptage** (théorique, dans un modèle précis) : au moins $\lceil m/32 \rceil$ `vpshufb` par paire de nombres et par m-vecteur ; A2 atteint cette limite pour $m \in \{80, 142\}$;
- **Correction et sûreté** : tests de réponses connues (KAT), tests unitaires, limites de lecture et d'écriture, propriétés des tables, indépendance vis-à-vis du bourrage ;
- **Mesures de temps** sur une machine physique (compteur TSC, unité statistique : le processus) : gains sur les noyaux seuls, puis sur `mayo_sign`, avec contrôle de l'effet d'ordre et des sondes de mesure ;
- **Analyse du code produit sans l'exécuter** (E1, E1b) : on compte les instructions et les écritures dans les boucles produites par GCC 16.2.0, et on désactive des passes du compilateur une à une ;
- **Chaque résultat porte une étiquette** : [F] fait établi, [I] déduction, [H] hypothèse non testée.

1.2 Revue de littérature

1.2.1 Implémentations existantes

Référence C. Arithmétique ordinaire, sans `vpshufb`.

AVX2 officiel (MAYO-C). Fichier `src/AVX2/shuffle_arithmetic.h` : macros `MULTIPLY_ACCUMULATE_M_VEC` et `DISENTANGLE_AND_ACCUMULATE_M_VECS`, avec le recouvrement décrit plus haut. C'est la base de ce travail : version amont `74de637` [15].

***Nibbling MAYO* [7].** Implémentation AVX2 et Cortex-M4 optimisée, avec la représentation en quartets. Au paragraphe 5.1, un `vpshufb` donne 64 multiplications, les accumulateurs impairs et pairs sont séparés et, pour $m = 96$ (MAYO-3 de l'époque), un recouvrement « duplique 32 opérations ». Le débit est d'un `vpshufb` par cycle sur Skylake et de deux sur Ice Lake. Nous avons lu l'article en entier. Il utilise les paramètres du tour 1 ($m = 64$ pour MAYO-1 et MAYO-2, 96 pour MAYO-3, 128 pour MAYO-5) : MAYO-1 et MAYO-5 n'y ont donc *pas* de fin de vecteur à traiter. Les mesures ont été faites sur Skylake (Xeon E3-1245 v5) et Ice Lake (Xeon Gold 6338), avec `clang-14`, Turbo Boost désactivé, médiane de 10 000 répétitions. Les tables de multiplication sont construites par la méthode de [14], appliquée deux fois puis entrelacée. L'approche par `vpshufb` se distingue de [14] en cherchant *deux* produits par recherche. L'article évoque le recouvrement pour m non multiple de 64 (cas MAYO-3), mais sans traitement spécial de la fin du vecteur. Le code publié avec l'article (`shuffle_arithmetic_\{64,96,128\}.h`) charge aux positions de limbes 0 et 2 (octets $[0, 32)$ et $[16, 48)$) et utilise quatre `vpshufb` par paire pour $m = 96$.

Version GFNI. Elle utilise l'instruction `vgf2p8affineqb` et lit sur toute la largeur dans une marge de sécurité (`MAYO_MVEC_SLACK`) ; elle est hors du cadre de ce mémoire.

1.2.2 Fuites par canaux auxiliaires et temps constant

Les attaques par mesure du temps [10] et par le cache [11] exigent que les adresses mémoire et les branches du programme ne dépendent pas de données secrètes. Dans les noyaux, les tables de multiplication calculées à partir du vinaigre ne sont que des données de `vpshufb` : elles ne servent ni d'adresses ni de conditions. A2 doit garder cette propriété ; la section 2.2 dit où en est la preuve.

1.3 Évaluation et positionnement

Le tableau 1.2 sépare ce qui était connu, ce qui est venu après l'article, et ce qu'A2 fait de différent. Il se fonde sur les sources consultées : le paragraphe 5.1 de l'article (lu en entier), les parties AVX2 du code, et tout l'historique du fichier `shuffle_arithmetic.h` dans le dépôt officiel (versions `3c9be9e`, `d81b689`, `11ff0f0`, `4b7cd94`, `fef05b5` et la dernière version de `main`, `74de637`).

Tab. 1.2 : Situation d'A2 par rapport aux sources consultées.

Catégorie	Élément	Source
Connu avant A2	`vpshufb` avec $t = 1$, deux produits par octet ; séparation impair/pair ; remise en ordre à la fin	[7]
Connu avant A2	Recouvrement de registres quand m n'est pas multiple de 64 (MAYO-3, $m = 96$)	[7]
Venu après l'article	Généralisation par chargements reculés pour tout $m \leq 192$	code officiel (tour 2)
Différent dans A2	Fin de vecteur $\leq$ 16 éléments regroupée dans *un* registre xmm (quartets [bas|haut]), avec *un* `vpshufb` xmm sur la voie basse de la table	ce travail
Différent dans A2	Variante SAFE pour $m = 142$: on charge $[63, 71)$ puis on décale, sans aucune lecture en dehors de l'entrée	ce travail
Différent dans A2	Propriétés vérifiées pour la fin du vecteur (section 2.2)	ce travail
Non revendiqué	Recouvrement, séparation impair/pair, remise en ordre tardive, schéma $t = 1$, nouveauté absolue	–

Nous n'avons trouvé aucun `vpshufb` xmm sur une fin de vecteur, ni dans le code publié avec l'article, ni dans l'historique officiel jusqu'à la dernière version de `main`. Les `__m128i` qui apparaissent à partir de `fef05b5` (lignes 548–606) servent à autre chose : transposer des m-vecteurs par décalage de bits, et charger une table de 16 octets répétée sur 256 bits. **Limite :** nous n'avons lu l'article en entier que pour sa section 5.1 et ses parties AVX2 utiles. Ne pas avoir trouvé la technique dans ces sources ne prouve pas qu'elle n'existe pas ailleurs.

Chapitre 2

Méthodologie

Ce chapitre décrit la démarche : la conception d'A2 et son modèle de comptage (section 2.1), les propriétés qui assurent sa correction et sa sûreté (section 2.2), puis le protocole d'expérience et d'analyse du code produit (section 2.3).

2.1 Conception d'A2 : comptage, mise en œuvre, hypothèses et limites

Cette section répond à QR1 et décrit la mise en œuvre (QR2). Les propositions 2.3 à 2.5 sont des énoncés mathématiques *dans un modèle* ; on n'en tire aucune conclusion sur la vitesse (section 2.1.4).

2.1.1 Modèle de comptage

Définition 2.1 (Données)**.** Un m-*vec* est un vecteur $v \in \mathbb{F}_{16}^m$, avec m pair, rangé sur $b = m/2$ octets : v_{2i} est le quartet bas de l'octet i et v_{2i+1} le quartet haut. Les K nombres $c_0, \ldots, c_{K-1}$ sont regroupés en $P = \lceil K/2 \rceil$ paires (complétées par 0 si K est impair). Pour chaque $k < K$, on calcule $\text{acc}_k \leftarrow \text{acc}_k \oplus c_k v$.

Pour la paire p, on pose $T_p[x] = (c_{2p}x) \oplus ((c_{2p+1}x) \ll 4)$. Un `vpshufb` de largeur $W \in \{16, 32\}$ octets fait W recherches indépendantes. Chacune donne, pour *un* élément de v, les deux produits de la paire.

Définition 2.2 (Classe $\mathcal{C}$)**.** Algorithmes qui (i) obtiennent chaque produit $c_k v_j$ par une recherche dans une table de 16 octets ; (ii) donnent à chaque recherche un quartet seul dans un octet (bit 7 nul) ; (iii) ne font pas deux recherches pour le même couple (élément, paire), sauf pour le recouvrement entre registres ; (iv) utilisent librement XOR, ET, décalages, permutations et chargements. Sont *hors* de $\mathcal{C}$: GFNI, les représentations *bitsliced*, les tables de 256 octets.

La quantité comptée est $N(\text{algo}, m)$: le nombre de `vpshufb` exécutés *par paire de nombres et par* m-*vec*.

2.1.2 Énoncés

Proposition 2.3 (Limite)**.** *Tout algorithme de $\mathcal{C}$ vérifie $N \geq \lceil m/32 \rceil$.*

Démonstration. Chaque élément v_j ($j < m$) demande au moins une recherche par paire, et une instruction en fait au plus $W \leq 32$. Donc $32N \geq m$. □

Proposition 2.4 (Schéma officiel avec recouvrement)**.** *Le schéma qui couvre les b octets avec $\lceil b/32 \rceil$ chargements de 32 octets (le dernier est reculé) et qui traite chaque registre avec deux* `vpshufb` *(impairs, pairs) a $N_{\text{off}}(m) = 2\lceil m/64 \rceil$.*

Démonstration. Il y a $\lceil b/32 \rceil = \lceil m/64 \rceil$ chargements. Chacun donne 32 indices impairs et 32 indices pairs, donc deux `vpshufb`. □

Proposition 2.5 (Mise en œuvre de la limite si $0 < r \leq 32$)**.** *Écrivons $m = 64q + r$ avec $0 \leq r < 64$, et $t = r/2$ le nombre d'octets de la fin du vecteur. Si $0 < r \leq 32$, l'algorithme suivant est dans $\mathcal{C}$ et a $N = 2q + 1 = \lceil m/32 \rceil$:*

1. *pour $g < q$: charger l'intervalle $[32g, 32g + 32)$ et le traiter avec deux* `vpshufb` *ymm ;*
2. *pour la fin du vecteur : charger les t octets de $[32q, 32q + t)$, construire X = [quartets bas des t octets | quartets hauts des t octets] (cela met les $r \leq 32$ indices dans un seul registre), puis appliquer* un seul `vpshufb` *;*
3. *remettre en ordre les sorties de la fin du vecteur (même formule que pour un registre complet, limitée aux r positions).*

Démonstration. On compte $2q+1$ instructions, et $\lceil m/32 \rceil = 2q + \lceil r/32 \rceil = 2q+1$ quand $0 < r \leq 32$. □

Conditions de mise en œuvre (le code doit les respecter ; le modèle ne les garantit pas) : (a) le chargement de la fin du vecteur ne lit que des octets de l'entrée ; (b) si $r \leq 16$ ($t \leq 8$), X tient dans un registre de 128 bits et on utilise la *voie basse* de T_p ; si $16 < r \leq 32$, X occupe un registre de 256 bits (voie basse = impairs, voie haute = pairs), ce qui demande que les deux voies soient égales ; (c) le coût de préparation de la fin du vecteur est payé une fois par entrée, et celui de la remise en ordre une fois par paire.

Corollaire 2.6 (Limite exacte)**.** *Pour $m \leq 192$, $\Delta(m) := N_{\text{off}}(m) - \lceil m/32 \rceil = 1$ si $0 < (m \bmod 64) \leq 32$, et 0 sinon. Dans le second cas, le code officiel atteint déjà la limite : aucun algorithme de $\mathcal{C}$ ne peut faire mieux en nombre de* `vpshufb`*.*

TAB. 2.1 : Application aux paramètres actuels.

	m	q	r	N_{off}	$\lceil m/32 \rceil$	Δ	Statut
MAYO-1	80	1	16	4	3	1	implémentée (A2), fin de 8 octets
MAYO-2	64	1	0	2	2	0	sans objet
MAYO-3	108	1	44	4	4	0	impossible dans $\mathcal{C}$
MAYO-5	142	2	14	6	5	1	implémentée (A2), fin de 7 octets + 1 de bourrage

Ce qui est fait et ce qui ne l'est pas. Le prototype traite les fins de vecteur d'au plus 16 éléments ($t \leq 8$, $r \leq 16$). La variante pour $16 < r \leq 32$ **n'est ni réalisée ni testée** : ce n'est pas un résultat acquis. Aucun des quatre paramètres actuels n'a $r \in (16, 32]$.

2.1.3 Conception d'A2

2.1.3.1 Structure du noyau

Pour une entrée de P_1 et une ligne d'accumulateurs de K vecteurs, A2 garde tels quels les $q = \lfloor m/64 \rfloor$ registres ymm complets sur les $32q$ premiers octets. Il traite ensuite la fin du vecteur ainsi :

1. **Préparation (une fois par entrée)** : chargement de 8 octets (`_mm_loadl_epi64`), décalage de 4 bits (`vpsrlw`), entrelacement `vpunpcklqdq` (quartets bas dans les octets 0 à 7, quartets hauts dans les octets 8 à 15), ET avec `0x0f`. Cela fait quatre instructions (cinq dans la variante SAFE, qui ajoute un décalage d'octets).
2. **Produits** : un `vpshufb` xmm avec la *moitié basse* de la table de multiplication déjà chargée, et un `vpxor` xmm dans un accumulateur xmm.
3. **Remise en ordre (fin de ligne)** : séparation impair/pair comme pour la partie ymm, puis lecture, modification et écriture d'*un seul* limbe de 64 bits (le limbe d'indice $4q = L - 1$, soit 4 pour MAYO-1 et 8 pour MAYO-5).

Dans l'assembleur produit par GCC 16.2.0, la boucle interne passe donc, par entrée, de 20 à 15 `vpshufb` pour MAYO-1 (10 ymm + 5 xmm) et de 36 à 30 pour MAYO-5 (24 ymm + 6 xmm).

2.1.3.2 Variante par défaut et variante SAFE (MAYO-5)

Pour MAYO-1, $b = 40 = 32 + 8$: la fin du vecteur est exactement la fin de l'entrée, $[32, 40)$. Pour MAYO-5, $b = 71$ et $64 + 8 = 72$:

- *par défaut* : lit $[64, 72)$, donc **un octet après** l'entrée (comme le noyau officiel, dont le dernier chargement couvre $[40, 72)$) ;
- *SAFE* : lit $[63, 71)$ puis décale le registre de 8 bits vers les poids faibles ; l'octet 63 est éliminé, les octets 64 à 70 passent aux positions 0 à 6, et la position 7 reçoit 0. **Toutes les lectures restent dans l'entrée.**

Si l'on intègre un jour A2, on ne retiendrait que la variante SAFE. Les quartets de bourrage (positions 142 et 143 de MAYO-5, car $16L = 144$) n'ont pas d'effet sur les positions utiles (section 2.2.4).

2.1.3.3 Préconditions

(P1) P_1 est rangé à plat, sans bourrage entre les entrées, à l'adresse $P_1 + e \cdot b$; (P2) les accumulateurs ont L limbes, avec du bourrage de quartets seulement à la fin du dernier limbe ; (P3) les tables de multiplication sont construites par la routine officielle (voies égales, lemme 2.7) ; (P4) $m \in \{80, 142\}$: les noyaux A2 ne sont pas actifs pour les autres paramètres (condition de compilation `MAYO_TAIL_XMM_ACTIVE`) ; (P5) compilation sans `ENABLE_PARAMS_DYNAMIC`.

2.1.4 La limite n'implique pas l'accélération

Faire moins de `vpshufb` ne garantit pas d'aller plus vite : il faut ajouter le coût de préparation de la fin du vecteur, les dépendances entre instructions, les rechargements depuis la pile, les chargements non alignés et le débit des unités de calcul. Si seul le débit du `vpshufb` (un par cycle sur Skylake [7]) limitait la boucle, le rapport de temps attendu serait N_{A2}/N_{off} : $15/20 = 0{,}750$ (MAYO-1) et $30/36 = 0{,}833$ (MAYO-5).

TAB. 2.2 : Rapport attendu par le seul débit de `vpshufb` et rapport mesuré (S1, A2/officiel, noyaux isolés).

Noyau	Limite « débit seul »	Mesuré (S1)	Lecture
MAYO-1 `P1_times_Vt`	0,750	0,780	plus lent que la limite
MAYO-1 `P1t_times_Vt`	0,750	0,710	plus rapide que la limite
MAYO-5 `P1_times_Vt`	0,833	0,807	plus rapide que la limite
MAYO-5 `P1t_times_Vt`	0,833	0,880	plus lent que la limite

Deux cas sur quatre sont plus rapides que ce que permettrait le seul comptage des `vpshufb` : d'autres effets jouent (par exemple moins de rechargements depuis la pile : de 11 à 5 pour MAYO-1 `P1_times_Vt`, de 30 à 24 pour MAYO-5), mais ce n'est pas établi. Le modèle fondé sur le débit n'explique donc pas à lui seul les gains. Une hypothèse de performance H-P (« sur un processeur où le débit des permutations est le facteur limitant, le temps est au moins $N \cdot P$ cycles par entrée ») ne peut se tester qu'avec des cycles d'horloge, pas avec des ticks TSC. Elle vient après ce travail et reste ouverte.

2.2 Propriétés de correction et de sûreté

Cette section répond à la partie « sûreté » de QR2. Chaque propriété porte une étiquette : **[C]** (déduite du code et de ses invariants, par relecture et par des assertions à la compilation), **[E]** (vérifiée par des tests) ou **[H]** (hypothèse sur la machine ou le compilateur). Une propriété **[E]** seule n'est pas une démonstration. Notation : $n_y = \lfloor m/64 \rfloor$ (1 pour MAYO-1, 2 pour MAYO-5), $T_{\text{OFF}} = 32n_y$ (32 ; 64), $T_{\text{LIMB}} = 4n_y$ (4 ; 8).

2.2.1 Limites de lecture de P_1

L'entrée e occupe les octets $[eb, (e+1)b)$.

- **[C]** Les n_y chargements ymm lisent $[eb+32g, eb+32g+32)$ pour $g < n_y$; comme $32n_y \leq b$ ($32 \leq 40$; $64 \leq 71$), ils restent dans l'entrée.
- **[C]** MAYO-1 : la fin du vecteur lit $[eb+32, eb+40)$, exactement la fin de l'entrée.
- **[C]** MAYO-5, SAFE : la fin du vecteur lit $[eb+63, eb+71)$ puis décale de 8 bits ; toutes les lectures restent dans l'entrée.

- [C] (variante par défaut, non retenue) MAYO-5 lit $[eb + 64, eb + 72)$: un octet après l'entrée. Pour $e < N_{\text{ent}} - 1$, c'est le premier octet de l'entrée suivante. Pour la dernière entrée, c'est l'octet 720 863 du tampon, qui est alloué (zone de $8{\cdot}$`P1_LIMBS_MAX`$= 731\,016$ octets). Des `_Static_assert` imposent $N_{\text{ent}}b + \text{OVERREAD} \leq 8{\cdot}$`P1_LIMBS_MAX` et OVERREAD$\leq 1$.
- [E] Test « garde-page » (la fin de P_1 est collée à une page interdite, sans marge) : le code officiel et A2 par défaut provoquent une erreur SIGSEGV pour MAYO-5, A2 SAFE non ; pour MAYO-1, aucun des trois. Ce test contrôle la propriété ci-dessus sans la démontrer.

Le noyau officiel de MAYO-5 lit lui aussi 1 octet de trop (son dernier ymm couvre les octets 40 à 71), et celui de MAYO-3 en lit 2 (son second ymm couvre $[24, 56)$ pour $b = 54$). A2 SAFE n'a besoin d'aucune marge.

2.2.2 Limites d'écriture

Pour la ligne r, l'accumulateur est à acc$+rKL$ (en limbes) ; le vecteur k occupe $[kL, (k+1)L)$.

- [C] Les écritures ymm touchent les limbes $[kL + 4g, kL + 4g + 4)$ pour $g < n_y$, et jamais au-delà du limbe $4n_y - 1 \leq L - 1$.
- [C] La fin du vecteur est traitée par lecture, modification puis écriture d'un seul limbe de 64 bits, d'indice $T_{\text{LIMB}} = L-1$, pour $k < K$ (et $k+1 < K$ pour le second, avec une garde explicite). Toutes les écritures sont donc dans les VKL limbes de `acc`. Le noyau officiel écrit un ymm à $L - 4$: même étendue.
- [E] Un « canari » placé après `acc` n'est pas modifié ; P_1, passé en `const`, n'est pas modifié non plus (la copie de l'entrée est inchangée).

2.2.3 Invariant des deux voies des tables

A2 lit la *voie basse* de chaque table et suppose qu'elle est égale à la voie haute.

Lemme 2.7. [C] *Pour tout i,* `V_multabs[i]` *a ses deux voies de 128 bits égales.*

Justification. (i) Chacune des quatre lignes de 32 octets de `__gf16_mulbase` répète deux fois la même suite de 16 octets (constantes lues dans `arithmetic_common.h`, lignes 12 à 17). (ii) `tbl32_gf16_multab2(b)` est un OU exclusif de ET entre ces lignes et des masques obtenus par `_mm256_set1_epi16(b & 0xf)` décalé puis comparé ; ces opérations agissent par élément de 16 bits, donc voie par voie, avec des masques identiques dans les deux voies. (iii) `mayo_V_multabs` combine deux telles tables par `^` et `_mm256_slli_epi16(.,4)`, aussi voie par voie. Par récurrence, l'égalité est conservée.

[E] Nous avons vérifié 272 tables avec le mode `MAYO_TAIL_XMM_CHECK` (arrêt en cas d'écart), sur les tables données aux noyaux. C'est une *condition préalable* d'A2 : si les tables étaient construites autrement (autre routine, autre version du code), il faudrait revérifier cet invariant.

2.2.4 Influence des quartets de bourrage

Seul MAYO-5 a des quartets de bourrage (positions 142 et 143 ; pour MAYO-1, $80 = 5 \cdot 16$).

- [C] *Chaque position est indépendante.* La sortie à la position j ne dépend que des entrées à la position j de chaque m-vec : une recherche lit un index et une table, jamais un autre index. Les décalages `srli/slli_epi16` mélangent deux octets voisins, mais chaque résultat est suivi d'un ET avec `0x0f` (ou vient d'un registre déjà masqué).
- [C] Les deux seules différences entre le code officiel et A2 aux positions de bourrage sont : pour l'officiel, les produits de l'octet 71 (premier octet de l'entrée suivante) ; pour A2 SAFE, 0.
- [C] Les noyaux alimentent `Pv`, puis `Vt_times_Pv` produit `VP1V`, qui est utilisé par `compute_rhs`. C'est le premier endroit qui mélange des positions, et il met à zéro les quartets de bourrage avant de les utiliser (`src/mayo.c`, lignes 50 à 60, masque $(1 \ll ((m \bmod 16) \cdot 4)) - 1$). Entre les noyaux et `compute_rhs`, toutes les opérations travaillent position par position.
- [E] Tests unitaires (20 000 cas par taille, 8 familles d'entrées) : 0 écart sur les $M/2$ octets utiles et sur les limbes complets après masquage. Sur 30 010 signatures appariées de MAYO-5 (15 005 pour MAYO-1 et pour MAYO-2), aucune signature n'est différente.

Limite. Cette propriété suppose que `Pv` n'est utilisé que par les chemins listés, ce que nous avons établi en cherchant dans le code source de la version AVX2. Si une modification future lisait le bourrage de `Pv`, il faudrait réexaminer la question.

2.2.5 Temps constant : ce qui est établi et ce qui ne l'est pas

- [C] Les bornes des boucles (V, K) et les adresses de chargement (eb) ne dépendent que de paramètres publics. Les tables calculées à partir du vinaigre ne sont que des données pour `vpshufb` : ce ne sont jamais des adresses ni des conditions. Le code officiel a la même structure.
- [C]→à vérifier sur le programme compilé : à la remise en ordre de la fin du vecteur, A2 ajoute deux `vmovq` d'un registre vectoriel vers un registre général (`ar[...] ^= (uint64_t)_mm_cvtsi128_si64(...)`). Des données liées au secret passent donc par un registre général, mais seulement comme donnée d'un XOR vers la mémoire. Nous avons inspecté l'assembleur (script `inspect_asm.py`) et n'avons trouvé aucun usage non conforme (adresse, branche, drapeau). Ce résultat vaut pour *une* version de GCC, pas en général.
- [E] (indice) Valgrind en mode temps constant (données secrètes marquées comme indéfinies), dans la VM seulement : il repère les branches et les adresses qui dépendent de données marquées, sur les exécutions testées.

- [H] Les instructions `vpshufb`, `vpxor`, `vpand`, les décalages, les chargements et les écritures prennent un temps qui ne dépend pas des données sur les processeurs visés (d'après la documentation du constructeur, non vérifiée ici).

Nous ne fournissons aucune preuve formelle de temps constant. Que les KAT réussissent et qu'il n'y ait pas de différence de résultat ne démontre pas le temps constant.

2.2.6 Correction fonctionnelle

Les KAT officiels réussissent pour MAYO-1, 2, 3 et 5 (toutes les sessions), ainsi que les tests du schéma et les tests unitaires précédents. Il y a zéro signature différente dans toutes les paires A/B, et le nombre de tentatives de signature est le même. La correction est donc établie *sur les entrées testées* ; l'argument général repose sur les propriétés [C] ci-dessus.

2.3 Protocole expérimental

L'objectif est qu'un lecteur qui a le dépôt, les archives et une machine comparable puisse refaire chaque analyse et, au bruit près, chaque mesure. Les commandes sont à l'annexe A.

2.3.1 Plateforme

Machine physique (toutes les mesures citées au chapitre 3). Portable Intel Core i9-9880H (8 cœurs, 16 threads, microcode `0x104`) ; `systemd-detect-virt` = `none`, drapeau hyperviseur = 0 ; Kali 7.1.5 (démarrage sur disque, double amorçage) ; GCC 16.2.0 (Debian 16.2.0-3), cmake 4.3.4, Python 3.14.8, numpy 2.4.6. Gouverneur `performance`, turbo actif, SMT actif, secteur branché. Processus épinglé sur le cœur 1 (`taskset`). La fréquence instantanée du cœur n'est enregistrée qu'au lancement (de 800 MHz au repos à 4 540 MHz selon les sessions) ; elle n'est pas une fréquence soutenue. Horloge du portable à UTC+2 ; les journaux `machine.txt` sont en UTC.

Environnement virtuel (historique, non utilisé pour les résultats de performance). VM KVM à 2 cœurs, GCC 13.3.0, cmake 3.28.3. Les mesures de cette VM (qui ont orienté les décisions de conception) ne sont jamais présentées comme performances matérielles.

Compilation. Options de la bibliothèque : `-O3 -funroll-loops -march=haswell -maes`, identiques pour les deux versions comparées. Pour l'analyse statique, la commande exacte de chaque objet est lue dans `compile_commands.json`.

2.3.2 Code comparé

Base amont : dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`. Les modifications sont appliquées par `git am` dans l'ordre : `0001` (prototype A2), `0002` (sondes de décomposition, inertes par défaut), `0003` (contrôle de l'effet des sondes), `0004` (analyse statique E1, aucun noyau modifié). Les identifiants de commit changent avec `git am` ; les arbres, non (annexe A). L'arbre des mesures S1/S2 est celui après `0001` ; celui de PH1/CT1 après `0003`. Les noyaux optimisés n'ont pas été modifiés pendant la phase d'analyse.

2.3.3 Protocole de mesure apparié

Pour chaque niveau mesuré (`P1_times_Vt`, `P1t_times_Vt`, `m_calculate`, `mayo_sign`), un processus exécute des paires (officiel, A2) dans un ordre *aléatoire*, avec la même graine et le même message pour les deux versions. Le temps est lu par `rdtscp` (ticks TSC). L'ordre d'exécution de chaque paire est enregistré (`xmm_first`). Un processus comprend 3 001 paires. Les signatures des deux versions sont comparées octet par octet, et le nombre de tentatives de signature est vérifié.

2.3.4 Unité statistique et intervalles

Les 3 001 paires d'un même processus ne sont pas des répétitions indépendantes (même état de la machine, même fréquence). L'intervalle à 95 % « naïf », calculé sur les paires, est d'environ $\pm 0{,}0001$ pour `mayo_sign`, alors que la dispersion entre processus est 5 à 15 fois plus grande. **L'unité statistique est le processus.** Pour chaque niveau et chaque processus, on calcule la médiane du rapport A2/officiel. La valeur donnée est la médiane de ces médianes, avec un intervalle à 95 % obtenu par *bootstrap sur les processus*. Nous testons une seule différence entre sessions (S2−S1) : avec deux démarrages, on ne peut pas estimer à part la variation « entre sessions » ; nous donnons donc l'écart et son intervalle bootstrap.

Effet d'ordre. Pour chaque processus, on calcule la médiane du rapport sur les paires « A2 en premier » et sur les paires « officiel en premier ». L'effet d'ordre est la différence, avec un intervalle bootstrap sur les processus ; on le juge par rapport au gain.

2.3.5 Sessions

Les scripts de lancement refusent d'écraser un dossier ou une archive existants : aucune campagne n'écrase la précédente. S1 et S2 comparent deux démarrages différents avec le même code, la même machine et le même compilateur. Pour les temps absolus, S2 est environ 7 % plus lent que S1 pour tous les niveaux, témoins compris ; nous n'avons pas mesuré la cause, et les rapports ne sont pas touchés.

Tab. 2.3 : Sessions de mesure sur machine physique (dates UTC).

Session	Date, démarrage	Plan	Objet
S1	19 oct. 21 :58–22 :43 ; un démarrage	4 campagnes × 5 processus × 3 001 paires (graines 1–20)	gains
S2	20 oct. 12 :10 ; nouveau démarrage	5 processus (graines 21–25)	reproductibilité
PH1	21 oct. 19 :13 ; nouveau démarrage	MAYO-5 : 10 processus ; MAYO-1, MAYO-2 : 5	décomposition instrumentée
CT1	21 oct. 19 :28 ; même démarrage que PH1	5 configurations × 10 processus (graines 41–50)	effet des sondes

2.3.6 Instrumentation de décomposition

Des sondes (`rdtscp` précédé d'un `lfence`) mesurent, pour chaque signature et chaque version, le temps total, le nombre de tentatives et, pour chaque phase, le temps et le nombre d'appels. Elles sont inactives par défaut : sans la macro correspondante, le code objet de `mayo.c` et de `arithmetic.c` est identique à l'original (vérifié par `cmp` et `objdump`). Le patch `0003` ajoute un masque pour activer chaque sonde séparément. Les cinq configurations de CT1 sont : `plain` (bibliothèque d'origine, aucune sonde), `none` (sondes compilées mais toutes désactivées), `esk` (sonde sur `expand_sk_internal` seule), `mcalc` (sonde sur `m_calculate_M_and_VPV` seule) et `all` (toutes les sondes).

2.3.7 Analyse statique du code généré (E1)

Pour chaque construction, nous recompilons l'objet avec la commande enregistrée, nous le désassemblons (`objdump -d -M intel`) et nous repérons les boucles de travail par leur nombre de `vpshufb` (36 ymm pour l'officiel de MAYO-5 ; 24 ymm + 6 xmm pour A2). Les comptes sont ramenés au facteur de déroulage de boucle (ici $u = 1$). Nous identifions le noyau par le *pas d'adresse sur* P_1 : `add reg,0x47` (entrées de 71 octets, le pas d'une entrée) pour `P1_times_Vt` et `imul ...,0x47` pour `P1t_times_Vt`. Les repères de débogage de GCC ne sont pas fiables (les corps identiques sont fusionnés et étiquetés `P1_times_O`). Pour attribuer une différence de code à une cause, il faut d'abord avoir écarté les autres modifications du code (section 2.3.8).

2.3.8 Protocole E1b (analyse statique de l'origine des écritures)

E1b recompile `src/mayo.c` avec *la commande exacte* de E1 plus une seule option en plus, sans aucune mesure de temps. Constructions : `P_none` (sans sondes), `P_all` (toutes les sondes), `TAB` (construction des mesures S1/S2). Variantes : `base` (contrôle : doit redonner exactement le code de E1) ; `dump` (fichiers de sortie `-fdump-tree-optimized`, `-fdump-tree-sra-details`, `-fdump-tree-esra-details`, `-fdump-rtl-expand`, `-fdump-rtl-ira`, `-fopt-info-inline-optimized` ; contrôle : le code doit être identique) ; `nosra` (`-fno-tree-sra`) ; `noipasra` (`-fno-ipa-sra`) ; `nodse` ; `nopre` ; `nofre`.

Pour chaque variante, le rapport donne : les boucles de travail avec leur nombre d'instructions et d'écritures de 32 octets ; le nombre d'instructions de `sign_signature_internal` *en dehors des boucles de travail* ; la distance entre les histogrammes de mnémoniques, hors boucles, par rapport à `base` ; et le nombre de lignes GIMPLE qui mentionnent `temp`. **Règle de lecture :** on ne lit jamais seule la disparition des écritures dans une variante. On ne l'attribue à une passe que si la distance hors boucles est faible, c'est-à-dire si rien d'autre n'a changé.

TAB. 2.4 : Issues possibles d'E1b et conclusions autorisées (fixées avant exécution).

	Observation (GCC 16.2.0)	Conclusion autorisée / non autorisée
1	Doublon absent avec `-fno-tree-sra` *et* distance en dehors des boucles faible	la passe conditionne le doublon dans cette compilation / non : que `temp[]` en soit la cause sans les fichiers d'analyse
2	Doublon absent mais distance élevée	l'attribution est *non établie*
3	Doublon présent dans toutes les variantes	les passes testées ne le suppriment pas, hypothèse non confirmée / non : que le doublon soit sans cause compilateur
4	`base` $\neq$ code de E1	E1 non reproduit : arrêter l'interprétation

2.3.9 Traçabilité et reproduction

Les données brutes (S1, S2, PH1, CT1) sont publiées avec leurs empreintes SHA-256 (annexe A). Le script de reproduction réextrait les archives, vérifie les empreintes, relance les neuf analyses et compare les sorties : **9 analyses sur 9 identiques** lors de la dernière exécution. On retrouve *exactement* les analyses ; pour les *mesures*, on obtiendra des valeurs proches mais pas identiques (ticks TSC, fréquence, bruit).

2.3.10 Menaces à la validité identifiées a priori

- **Une machine, un compilateur sur machine physique**, trois démarrages ; S2 sur 5 processus.
- **Ticks TSC, non cycles cœur** : la fréquence d'exécution réelle est inconnue ; aucune conclusion sur le débit des unités d'exécution n'est possible.
- **Effet d'ordre** documenté (section 3.3).
- **Les sondes modifient le code** : mesures instrumentées $\neq$ mesures sans sonde.
- **Pas de compteurs matériels** disponibles dans l'environnement d'analyse.
- **Première exécution de la campagne s3 de S1** : CSV bruts non archivés (seule la seconde exécution l'est).
- **Comparaison VM/physique** : le gain de MAYO-5 sur `mayo_sign` était de 7,1 % dans la VM (GCC 13.3.0) ; l'écart avec la machine physique mêle machine et compilateur et n'est pas expliqué.

Chapitre 3

Résultats, interprétation et discussion

Sauf indication contraire, les valeurs sont des rapports A2 SAFE / officiel (médiane des processus), en ticks TSC, sur la machine physique décrite au chapitre 2. Les étiquettes [F], [I], [H] sont celles de l'introduction générale : [F] fait, [I] déduction, [H] hypothèse.

3.1 Correction

[F] Les tests officiels (KAT) de MAYO-1, 2, 3 et 5 réussissent dans toutes les sessions. Il y a 0 signature différente sur les 30 010 paires de MAYO-5 et sur les 15 005 paires de MAYO-1 et de MAYO-2, et le nombre de tentatives est le même dans les deux versions. Les propriétés de sûreté sont dans la section 2.2.

3.2 Gains sur les noyaux et sur la signature (S1, S2)

Tab. 3.1 : Rapport A2/officiel, médiane des processus [IC95 bootstrap entre processus]. S1 : 20 processus ; S2 : 5 processus. Gain = 1 − rapport.

Paramètre	Niveau	S1	S2	Gain S1 / S2
MAYO-1	`P1_times_Vt`	0,7803 [0,7793 ; 0,7817]	0,7814 [0,7794 ; 0,7832]	22,0 / 21,9 %
	`P1t_times_Vt`	0,7099 [0,7060 ; 0,7124]	0,7080 [0,7024 ; 0,7126]	29,0 / 29,2 %
	`m_calculate`	0,8110 [0,8096 ; 0,8118]	0,8102 [0,8080 ; 0,8123]	18,9 / 19,0 %
	`mayo_sign`	**0,8975** [0,8970 ; 0,8979]	**0,8974** [0,8968 ; 0,8990]	**10,3 / 10,3 %**
MAYO-5	`P1_times_Vt`	0,8067 [0,8060 ; 0,8071]	0,8049 [0,8043 ; 0,8076]	19,3 / 19,5 %
	`P1t_times_Vt`	0,8798 [0,8783 ; 0,8818]	0,8792 [0,8770 ; 0,8858]	12,0 / 12,1 %
	`m_calculate`	0,8851 [0,8845 ; 0,8863]	0,8869 [0,8847 ; 0,8874]	11,5 / 11,3 %
	`mayo_sign`	**0,9873** [0,9871 ; 0,9876]	**0,9871** [0,9863 ; 0,9876]	**1,3 / 1,3 %**

[F] Pour les témoins (aucun noyau A2), le rapport vaut 1 : MAYO-2 `mayo_sign` 1,0000 / 1,0000 et `m_calculate` 1,0001 / 0,9996 ; MAYO-3 `mayo_sign` 1,0000 / 1,0001. Les écarts entre S2 et S1 sur les huit lignes de gains sont au plus de 0,0019 en valeur absolue,

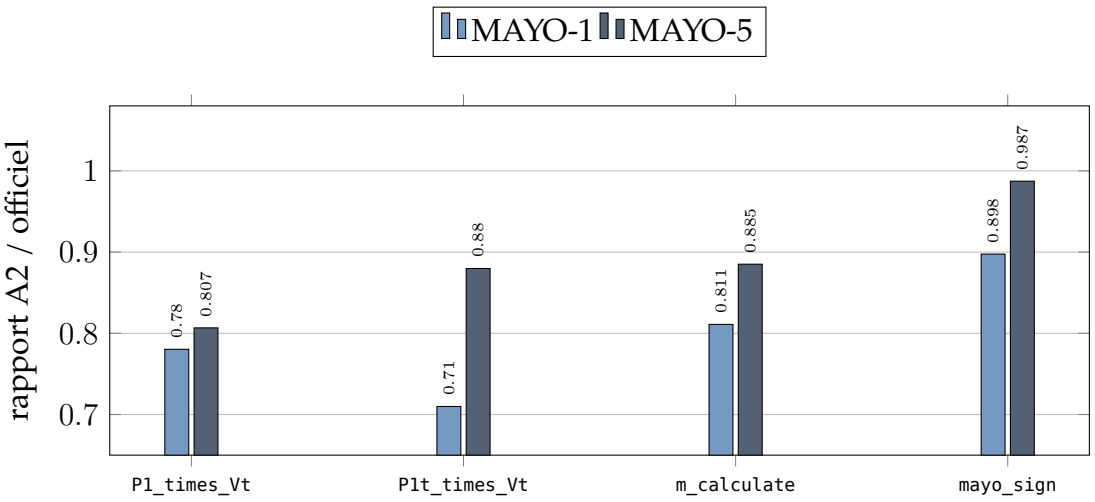


Fig. 3.1 : Rapports A2/officiel mesurés (S1). Le gain se dilue de la couche noyau à la signature complète, fortement pour MAYO-5.

et l'intervalle à 95 % contient 0 (par exemple pour `mayo_sign` : $-0{,}0001$ $[-0{,}0009\,;\,+0{,}0016]$ pour MAYO-1 et $-0{,}0003$ $[-0{,}0011\,;\,+0{,}0003]$ pour MAYO-5).

Variabilité à quatre niveaux. Écart-type robuste entre les paires d'un même processus : 0,0015 (S1) et 0,0013 (S2) pour MAYO-1, 0,0114 et 0,0039 pour MAYO-5. Entre les processus d'une même session : 0,0009 (20 processus) et 0,0010 (5 processus) pour MAYO-1, 0,0008 et 0,0005 pour MAYO-5. Entre les campagnes de S1 : 0,0001 et 0,0002. C'est donc la dispersion entre processus, et non entre paires, qui fixe l'incertitude.

Gain de MAYO-5 sur la signature. [F] Il est de 1,3 % dans S1, S2, PH1/CT1 et dans la mesure de référence, proche du critère d'environ 1 % fixé avant les mesures. C'est le seul chiffre de gain de signature de MAYO-5 à retenir (section 3.4).

3.3 Effet d'ordre

[F] Les ordres sont équilibrés : A2 passe en premier dans 49,97 % des paires de `P1t_times_Vt` de MAYO-1 en S1 ($29\,993/60\,020$) et dans 50,5 % en S2 ; la probabilité que A2 passe en premier, sachant que A2 était le précédent, est de 0,5003.

[F] Le protocole produit lui-même un effet général d'environ 0,1 % (visible sur MAYO-2 `m_calculate`, où les deux versions sont le même code). L'effet de MAYO-1 `P1t_times_Vt` est environ quinze fois plus grand et propre à ce noyau. Il n'a pas d'effet sur `mayo_sign` de MAYO-1 ($\leq 0{,}0001$). Pour `mayo_sign` de MAYO-5, il vaut au plus $0{,}0006$, soit moins de 5 % du gain de $0{,}0127$. La version exécutée en second est plus rapide (MAYO-1 `P1t_times_Vt`, S1 : officiel $-0{,}9$ %, A2 $-1{,}6$ %), et l'effet est plus fort pour A2, d'où l'écart sur le rapport. [H] La cause n'est pas démontrée et nous n'avons fait aucune expérience pour la chercher. Pour ce noyau, nous donnons aussi la moyenne des deux ordres (0,706) en plus de la médiane de l'ensemble (0,710).

Tab. 3.2 : Rapport A2/officiel selon l'ordre d'exécution, et écart par processus [IC95].

Niveau	Session	A2 en 1er	officiel en 1er	écart [IC95]
MAYO-1 `P1t_times_Vt`	S1	0,7142	0,6985	+0,0151 [+0,0101 ; +0,0206]
	S2	0,7095	0,7009	+0,0043 [−0,0070 ; +0,0161]
MAYO-1 `mayo_sign`	S1	0,8975	0,8975	−0,0001 [−0,0002 ; 0,0000]
MAYO-5 `mayo_sign`	S1	0,9870	0,9876	−0,0006 [−0,0008 ; −0,0005]
	S2	0,9871	0,9871	0,0000 [−0,0002 ; 0,0000]
MAYO-5 `P1_times_Vt`	S1 / S2	0,8071 / 0,8053	0,8059 / 0,8043	+0,0010 / +0,0010
MAYO-2 `m_calculate` (témoin)	S1 / S2	1,0005 / 1,0004	0,9997 / 0,9994	+0,0010 / +0,0006

3.4 L'anomalie de MAYO-5 : décomposition instrumentée et contrôle

3.4.1 Toutes sondes actives (PH1)

[F] MAYO-5 : il y a une tentative de signature par signature dans les deux versions ; chaque phase est appelée une fois par signature, dont `P1_times_Vt` et `P1t_times_Vt` une fois par tentative. Le temps total est de 1 374 501 ticks (officiel) et 1 274 140 (A2) : rapport 0,9270 [0,9253 ; 0,9283], soit un gain de 7,3 %. La différence moyenne de −100 967 ticks se répartit ainsi : −100 529 pour `m_calculate` (99,6 %), dont −54 584 pour `P1_times_Vt` et −46 166 pour `P1t_times_Vt`. Les autres phases (dont `expand_sk_internal`, `sample_solution`, `compute_A`) sont identiques à ±0,1 % près. MAYO-1 : rapport 0,8983 ; MAYO-2 (témoin) : 0,9999.

3.4.2 Le gain avec toutes les sondes n'est pas le gain sans sonde (CT1)

Tab. 3.3 : Cinq configurations du même programme (10 processus chacune, graines 41–50, un démarrage). Ticks par signature.

Configuration	Officiel	A2	Rapport [min–max]	Gain
`plain` (aucune sonde)	1 504 741	1 484 504	0,9871 [0,9862–0,9879]	1,29 %
`none` (sondes compilées, désactivées)	1 504 802	1 484 611	0,9868 [0,9860–0,9880]	1,32 %
`esk` (`expand_sk_internal`)	1 504 167	1 484 146	0,9871 [0,9861–0,9879]	1,29 %
`mcalc` (`m_calculate_M_and_VPV`)	1 502 948	1 483 529	0,9877 [0,9865–0,9885]	1,23 %
`all` (toutes les sondes)	1 372 740	1 273 970	0,9276 [0,9261–0,9291]	7,24 %

3.4.3 Mesure directe, sans soustraction de médianes (config. `mcalc`)

[F] Le reste de la signature est identique dans les deux versions. Toute la différence est dans `m_calculate` *quand il est dans la signature et sans sonde interne* : il coûte 931 000 ticks (officiel) et 912 000 (A2), contre environ 801 000 et 707 000 quand il est isolé ou avec

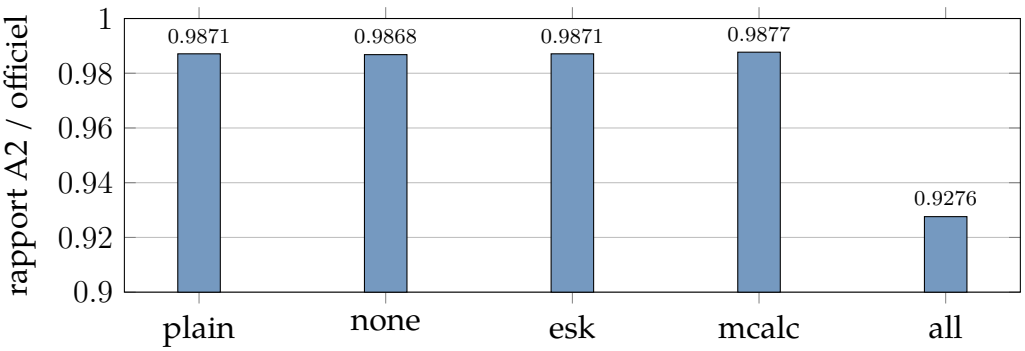


Fig. 3.2 : MAYO-5, rapport `mayo_sign` A2/officiel selon la configuration de sondes (CT1). Seule la configuration « toutes les sondes » change le résultat.

Tab. 3.4 : Décomposition directe de la signature MAYO-5 (ticks par signature).

	Officiel	A2	Différence
`m_calculate` dans la signature	931 453	912 476	$-18\,406$ [$-19\,396$; $-17\,558$]
hors `m_calculate` (total − sonde)	566 917	567 323	$+406$ (0,07 %)
`expand_sk_internal` (config. `esk`)	322 655	322 661	$+6$
`m_calculate` isolé (banc, tampons réutilisés)	801 324	707 314	$\approx -94\,000$ (11,7 %)
`m_calculate`, toutes sondes, dans la signature	801 073	699 775	$\approx -101\,000$

toutes les sondes. Le surcoût dû au contexte est d'environ 130 000 ticks pour l'officiel et 205 000 pour A2. La différence de 75 000 ticks entre les deux versions absorbe donc l'essentiel du gain mesuré sur le noyau isolé. Ce surcoût disparaît avec toutes les sondes (donc avec des `lfence` entre les sous-étapes de `m_calculate`), mais pas avec une seule sonde autour de `m_calculate`.

Correction d'une interprétation antérieure. Nous avions d'abord lu : « le reste de la signature ralentit avec A2 ». C'était une erreur, venue d'une soustraction de médianes prises à des niveaux de mesure différents. Cette lecture est *retirée*.

3.5 Analyse statique du code généré (E1, GCC 16.2.0)

3.5.1 Comptage de base

[F] Par entrée de P_1, avec le déroulage de boucle ramené à 1, le code officiel a 20 `vpshufb` (MAYO-1) et 36 (MAYO-5), et A2 en a 15 (10 ymm + 5 xmm) et 30 (24 ymm + 6 xmm), comme le prévoit le modèle de la section 2.1. Dans le banc de test, la boucle interne passe de 124 à 106 instructions pour MAYO-5 `P1_times_Vt` et de 126 à 110 pour `P1t_times_Vt` ; pour MAYO-1, de 68 à 51 et de 71 à 56. Les rechargements depuis la pile passent de 30 à 24 (MAYO-5 `P1_times_Vt`) et de 11 à 5 (MAYO-1). Cette réduction est une explication appuyée sur l'assembleur ; ce n'est pas une preuve que ces instructions expliquent à elles seules le gain.

3.5.2 Écritures supplémentaires en contexte de signature

Tab. 3.5 : Instructions de la boucle interne par entrée de P_1 (MAYO-5) selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
banc isolé	124	106	126	110
contexte, toutes sondes	123	105	126	109
contexte, sans toutes les sondes (`ctx_ab`, `esk`, `mcalc`, `none`)	123	135	162	139

[F] La différence entre « sans toutes les sondes » et « toutes les sondes » est *exactement* une écriture de 32 octets (`vmovdqa [rbp-...], ymm`) de plus par accumulateur et par entrée : +36 pour `P1t_times_Vt` officiel (36 accumulateurs) et +30 pour chaque noyau A2 (30 accumulateurs). Le reste de la boucle est identique, instruction pour instruction. La boucle officielle `P1_times_Vt` n'a pas d'écriture en double. Ces écritures vont vers une deuxième série d'emplacements consécutifs (pas de `0x20`), situés à `0xbfc0` octets de la première, dont l'adresse est prise ailleurs dans la fonction (`lea`). Deux boucles de 160 instructions (`Ut_times_O`, `m_lambda_times_O`) ont aussi une écriture en double (de 123 à 160) dans *les deux* versions. Le cadre de pile dépasse 220 Ko : on ne peut donc pas distinguer, par le seul décalage, un débordement de registres d'un tableau local. Avec GCC 13.3 (dans la VM), aucune écriture en double n'apparaît. Pour MAYO-1 (GCC 16.2.0), elle ne touche que `P1t_times_Vt` (officiel de 71 à 91 ; A2 de 56 à 71).

[F] Nous repérons la boucle officielle `P1_times_Vt` dans le contexte par son pas d'adresse `add reg,0x47`, et nous avons contrôlé ce repérage dans six constructions. Elle compte 123 instructions et 29 écritures dans les contextes officiel, sans sondes et avec toutes les sondes (à des adresses distinctes), sans écriture en double. Le tableau 3.5 résume ces comptes.

3.6 Lien quantitatif entre instructions et temps : modèle à un paramètre

[F] Gain calculé sur le code (P1+P1t) : 13,6 % sur le banc, 14,1 % dans le contexte avec toutes les sondes, et **3,9 %** dans le contexte sans sondes (250 → 216 ; 249 → 214 ; 285 → 274 instructions). Gain mesuré de `m_calculate` : 11,5 à 11,7 % isolé, 12,4 % avec toutes les sondes, **2,1 %** sans sondes (936 031 → 916 206). Dans les trois cas, le gain calculé sur le code et le gain mesuré évoluent dans le même sens.

Modèle. On suppose que le temps d'une boucle est proportionnel à son nombre d'instructions par entrée, avec un même coefficient c pour les deux versions ($N_{\text{ent}} = 10\,153$). Calé sur « toutes les sondes », on obtient $c = 635\,681/(249 \cdot 10\,153) = 0{,}251$ tick par instruction. **Prévision** sans sondes : un gain de $28\,082$ ticks pour P1+P1t ; **mesure** : $19\,825$. Le modèle *exagère* donc le gain d'environ 40 %.

Coût d'une écriture en plus. À partir de la différence entre « sans sondes » et « toutes sondes », on trouve $130\,895/365\,508 = 0{,}358$ tick par écriture pour l'officiel et $210\,808/609\,180 = 0{,}346$ pour A2.

Portée de ce rapprochement. Les valeurs 0,358 et 0,346 sont proches, mais c'est seulement un test de cohérence (un paramètre, deux observations) ; *ce n'est pas une validation du modèle*. Le modèle exagère le gain : il n'explique donc pas l'écart de façon chiffrée. Le fait que les écritures en plus et la perte de gain évoluent ensemble ne prouve pas que l'un cause l'autre.

3.7 Analyse statique E1b : résultats (GCC 16.2.0)

Nous avons exécuté E1b sur la machine de mesure (GCC 16.2.0, binutils 2.47). L'archive des sorties (SHA-256 `49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa`) est publiée avec le script `e1b_dumps_analyse.py`, qui produit les comptes cités ici. Aucune mesure de temps n'est faite.

3.7.1 Contrôles

[F] Pour les trois constructions (`P_none`, `P_all`, `TAB`), le code de `sign_signature_internal` recompilé en variante `base` est identique à celui d'E1. La variante `dump` (qui ajoute seulement des fichiers de sortie) donne aussi un code identique (distance 0 en dehors des boucles). Les constructions `TAB` et `P_none` donnent les mêmes comptes dans toutes les variantes.

3.7.2 Désactivation de passes, une à la fois

[F] Aucune des variantes `nosra`, `noipasra`, `nodse`, `nopre` (qui désactivent SRA, IPA-SRA, DSE et PRE) ne supprime l'écriture en double dans `P_none` ni dans `TAB` : les comptes des boucles ne changent pas (ligne 3 du tableau 2.4). [F] Avec la variante `nofre`, les boucles A2 gardent leur écriture en double, mais le code change beaucoup ailleurs (distance 357) et les boucles officielles prennent une autre forme. Dans `P_all`, la même option *fait apparaître* l'écriture en double dans les boucles A2 (135/52, 139/52), avec une distance de 876. [I] Ces deux dernières observations ne permettent *aucune attribution* à la passe FRE, puisque d'autres parties du code changent en même temps. [F] Avec la variante `noipasra` sur `P_all`, la structure de la fonction change tellement qu'aucune boucle de travail n'est reconnue : on ne peut rien lire.

3.7.3 Ce que montrent les sorties de GCC

SRA ne touche pas à `temp`, dans aucune des deux constructions. [F] Dans le fichier d'analyse `sra`, `temp` est écarté 12 fois dans chacune (8 fois pour « adresse prise hors d'un argument d'appel » et 4 fois pour « accès absents ou qui se recouvrent »). Le fichier `esra` n'en parle pas. SRA ne peut donc pas expliquer la différence entre

Tab. 3.6 : E1b : boucles de travail selon l'option ajoutée (instructions / écritures de 32 ou 16 octets). « Dist. » : distance entre histogrammes de mnémoniques en dehors des boucles de travail, par rapport à `base`.

Constr.	Variante	Dist.	`P1` A2	`P1t` A2	`P1t` officiel
`P_none`	`base`	0	135 / 52	139 / 52	162 / 65
	`nosra`	1	135 / 52	139 / 52	162 / 65
	`noipasra`	1	135 / 52	139 / 52	162 / 65
	`nodse`	412	135 / 52	139 / 52	162 / 65
	`nopre`	17	135 / 52	139 / 52	162 / 65
	`nofre`	357	135 / 52	139 / 52	185 / 58 (autres boucles modifiées)
`P_all`	`base`	0	105 / 22	109 / 22	126 / 29
	`nosra`	3	105 / 22	109 / 22	126 / 29
	`nodse`	45	105 / 22	109 / 22	126 / 29
	`nopre`	5	105 / 22	109 / 22	126 / 29
	`nofre`	876	**135 / 52**	**139 / 52**	185 / 58
	`noipasra`	1181	aucune boucle de travail reconnue (fonction de 2 892 instr.)		

`P_none` et `P_all`, ce qui correspond au tableau 3.6. L'hypothèse posée avant E1b (rôle de SRA) est *écartée*.

Les tableaux sont présents dans les deux constructions. [F] Le code GIMPLE optimisé déclare les mêmes tableaux locaux dans les deux cas (7 `__m256i temp[36]`, 2 `__m256i temp[24]`, 2 `__m128i tt[6]`).

Ce qui diffère : les écritures dans la boucle. [F] Dans le GIMPLE optimisé, 8 blocs de travail contiennent des écritures `temp[i] = ...` (ou `tt[i] = ...`) *dans le corps de la boucle* pour `P_none`, contre 3 pour `P_all`. Dans ces blocs, le nombre d'écritures par tour est égal au nombre d'accumulateurs (24 + 6 pour A2, 36 pour l'officiel). La boucle ne *lit jamais* `temp[]`, et les accumulateurs passent d'un tour à l'autre par des variables PHI (`prephitmp_N`). Dans `P_all`, 9 blocs utilisent des variables `temp_I_lsm_N` sans écrire le tableau (144 variables `temp_I_lsm` sont déclarées dans `P_none`, 300 dans `P_all`).

Lien avec l'assembleur. [F] La boucle A2 de `P1_times_Vt` compte 52 écritures dans `P_none` (42 de 32 octets et 10 de 16 octets) et 22 dans `P_all` (18 et 4). Les 22 écritures de `P_all` visent exactement les mêmes emplacements de pile que 22 des 52 de `P_none`. Les 30 autres sont 24 écritures de 32 octets (pas de `0x20`) et 6 de 16 octets (pas de `0x10`), ce qui correspond à la forme de `temp[24]` et de `tt[6]`. [I] Nous identifions ces 30 écritures en plus aux écritures de `temp[]/tt[]` du GIMPLE à partir de ce décompte et de la forme des tableaux, et non d'une correspondance entre noms de symboles.

3.7.4 Interprétation autorisée

[I] Dans la construction sans toutes les sondes, GCC remplace les lectures de `temp[]` dans la boucle par des valeurs qui passent d'un tour à l'autre dans des PHI, mais il *ne*

retire pas les écritures de la boucle. Le tableau, dont l'adresse est utilisée après la boucle, est donc mis à jour à chaque tour, en plus des emplacements où les accumulateurs débordent en mémoire. Dans la construction avec toutes les sondes, les accumulateurs sont traités comme des variables simples (suffixe `_lsm`) et le tableau n'est écrit qu'à la sortie de la boucle. [I] Cela correspond à l'hypothèse de départ (`temp[]` gardé en mémoire en plus des variables), mais *sans que SRA y soit pour quelque chose*.

[H] Selon la convention de nommage de GCC (que nous n'avons pas revérifiée dans ses sources), le suffixe `_lsm` correspond au *déplacement des écritures hors des boucles* (passe `lim`) et le préfixe `prephitmp` à la passe PRE. Ces rapprochements doivent être confirmés par les fichiers d'analyse de ces passes, que E1b n'a pas produits. **Non établi :** pourquoi ce déplacement a lieu dans `P_all` et pas dans `P_none`, quelle passe en décide, et si l'écriture en double disparaît sans autre changement du code. Le lien entre ces écritures et la perte de gain mesurée reste, comme à la section 3.6, une simple corrélation.

3.8 Comparaison avec la VM

[F] Dans la VM (Xeon virtualisé, GCC 13.3.0), le gain de `mayo_sign` pour MAYO-5 était de 7,1 %, proche des 7,3 % mesurés avec toutes les sondes sur la machine physique. [H] L'écart avec les 1,3 % sans sonde mélange l'effet de la machine et celui du compilateur ; il n'est pas expliqué. Nous le mentionnons sans nous en servir.

3.9 Discussion critique

Cette section sépare ce qui est établi, ce qui est déduit et ce qui reste hypothèse, puis discute les limites et les pistes abandonnées.

3.9.1 Bilan : établi, déduit, supposé

3.9.2 Pourquoi le gain se dilue

Le gain d'un noyau ne se retrouve sur la signature qu'à proportion de son poids. Pour MAYO-5 sans sondes, `m_calculate` pèse environ $931\,453/1\,504\,741 \approx 62\,\%$ de la signature, et son gain dans la signature est de $18\,406$ ticks, soit $1{,}2\,\%$ de la signature : c'est, à la précision des mesures, le gain de 1,3 % observé. Pour MAYO-1, le rapport entre le gain de la signature (10,3 %) et celui de `m_calculate` (18,9 %) donne un poids d'environ 54 % [I]. La vraie question n'est donc pas la part de la signature occupée par `m_calculate`, mais pourquoi son gain tombe de 11,7 % (isolé) à 2,0 % (dans la signature, sans sondes) pour MAYO-5, alors qu'il reste de 18,9 % pour MAYO-1 [F].

3.9.3 Ce que la concordance statique/dynamique prouve et ne prouve pas

Le désassemblage donne un fait solide : le nombre d'instructions de la boucle change selon la construction, et pas de la même façon dans les deux versions. Les temps varient dans le même sens. Il faut cependant trois précautions.

1. **Un modèle à un seul paramètre qui exagère le gain de 40 %** ne peut pas servir de validation. Le fait que le coût d'une écriture soit proche dans les deux versions (0,358 contre 0,346) est un contrôle de cohérence, pas une preuve.
2. **Corrélation, pas cause** : les constructions qui diffèrent par l'écriture en double diffèrent aussi par la présence de `lfence` entre les sous-étapes, par la taille de la fonction et par la répartition de la pile. Seule une expérience qui supprime l'écriture en double *sans autre changement du code* permettrait de conclure (voir la conclusion générale).
3. **Ticks et non cycles** : un « coût par écriture » de 0,35 tick correspond à un ordre de grandeur de 0,5 à 0,65 cycle si la fréquence réelle est de 3,5 à 4,5 GHz (non mesurée) pour un TSC à 2,4 GHz. Que cela soit compatible avec une écriture par cycle est une remarque [H], pas un résultat.

Plusieurs raisons peuvent expliquer que le modèle exagère le gain (toutes [H]) : le coût des instructions n'est pas le même pour toutes (un `vpshufb` et une écriture ne coûtent pas pareil) ; d'autres boucles que P1/P1t ont aussi une écriture en double (`Ut_times_O`, `m_lambda_times_O`), sans que l'on connaisse ici leur nombre de tours ; et des chargements sur la même ligne de cache peuvent se recouvrir.

3.9.4 Pourquoi MAYO-1 et MAYO-5 se comportent différemment

[I] Pour MAYO-1, l'écriture en double ne touche que `P1t_times_Vt`, dans les deux versions et à un rapport voisin ($71/91 = 0{,}78$ contre $56/71 = 0{,}79$ dans le banc). L'avantage calculé sur le code est gardé (0,770 sur le banc, 0,774 dans le contexte), ce qui va avec un gain de signature qui ne s'effondre pas. Pour MAYO-5, l'écriture en double touche les deux noyaux d'A2 mais un seul noyau de l'officiel. Cette lecture est compatible avec les faits, mais elle reste une déduction, pour la raison donnée plus haut.

3.9.5 Pistes explorées puis écartées

Nous avons essayé les pistes suivantes dans la VM avant de passer à la machine physique. Nous les gardons ici comme des hypothèses éliminées, avec leurs limites.

Fusion de `P1_times_Vt` et `P1t_times_Vt` en un seul parcours. Idée : lire P_1 une seule fois au lieu de deux. Le nombre de lectures d'entrées est bien divisé par deux (MAYO-1 : 6 480 → 3 240 ; MAYO-5 : 20 306 → 10 153). Mais, avec des accumulateurs

simultanés (option 1), le temps de `mayo_sign` augmente de 4,9 à 10,4 % (MAYO-1) et de 5,3 à 7,0 % (MAYO-5). Avec fermeture immédiate (option 2), il augmente de 67 à 78 % (MAYO-1, MAYO-3, MAYO-5). L'option 1 demande $V_{\max}$ accumulateurs, soit de +49 Kio (MAYO-1) à +159 Kio (MAYO-5) de pile. La correction est vérifiée (KAT et 5 000 essais indépendants par paramètre, sans écart). Interprétation : le travail SIMD ne change pas, et la pression sur la mémoire due aux accumulateurs dispersés l'emporte sur la baisse du nombre de lectures. Piste close ; mesures faites dans la VM seulement.

Transposée de P_1. Un diagnostic sur la représentation de P_1 et sur la façon d'y accéder conclut que la disposition en mémoire joue un rôle important dans le gain théorique maximal de 29 à 31 % observé sur MAYO-3 et MAYO-5. Nous ne pouvons pas dire quelle part vient du cache (pas de compteurs matériels). Le coût de construction de la transposée l'emporte sur le gain. Piste close.

Traitement par lots dans EF. Piste fermée ; le patch, le banc et les données sont archivés. Nous avons relevé une anomalie sur MAYO-2 : le rapport mesuré à un seul niveau change d'amplitude selon l'état du processus, ce qui impose de lire les rapports en tenant compte de la dérive de la phase concernée.

3.9.6 Limites de la validation

Une seule machine ; un seul compilateur sur la machine physique ; trois démarrages ; S2 sur cinq processus ; un gain de MAYO-5 proche du critère de 1 % ; des sondes complètes qui changent le comportement du code ; pas de cycles d'horloge ; pas de compteurs matériels ; Clang non examiné ; aucune preuve formelle de temps constant ; des propriétés de bourrage qui reposent sur une liste des utilisations de `Pv`. De plus, les mesures de l'article de référence ont été faites avec `clang-14` sur Skylake et Ice Lake, avec les paramètres du tour 1 : elles ne sont pas directement comparables aux nôtres (GCC 16.2.0, i9-9880H, paramètres du tour 2).

3.9.7 Sur l'intégration dans MAYO-C

Nous ne prenons ici aucune décision d'intégration. Voici ce qu'il faut peser : le gain de 10,3 % pour MAYO-1 (solide sur deux démarrages) ; le gain de 1,3 % pour MAYO-5 (proche du critère et dépendant du code produit par le compilateur) ; les conditions à respecter (deux voies égales, $m \in \{80, 142\}$) ; l'état de la preuve de temps constant (des indices seulement) ; et la dépendance au compilateur. Pour décider avec plus de solidité, il faudrait au moins une deuxième machine ou un deuxième compilateur.

Tab. 3.7 : Ce qui est établi, déduit ou supposé.

Statut	Énoncé	Appui
[F]	Nombre de `vpshufb` par entrée : 20→15 (MAYO-1), 36→30 (MAYO-5)	assembleur, preuve (prop. 2.5)
[F]	Gains de noyaux 12–29 %; `mayo_sign` 10,3 % (MAYO-1), 1,3 % (MAYO-5) sur cette machine et ce compilateur	S1, S2, PH1, CT1
[F]	Le reste de la signature est identique entre bras ; la différence est dans `m_calculate` en contexte	mesure directe (`mcalc`)
[F]	Avec toutes les sondes, le gain de MAYO-5 est 7,3 % ; sans, 1,3 %	CT1
[F]	GCC 16.2.0 ajoute une écriture de 32 octets par accumulateur quand toutes les sondes ne sont pas actives, et pas de la même façon dans les deux versions	désassemblage E1
[I]	Les écritures en plus, inégales entre les deux versions (+60 pour A2, +36 pour l'officiel sur P1+P1t), expliquent l'essentiel de la perte de gain de MAYO-5 dans la signature	accord entre le code et les mesures
[H]	Le coût vient du nombre d'écritures que le processeur peut faire par cycle, plutôt que des dépendances entre écriture et lecture ou des conflits d'adresses	non testé
[F]	Dans le GIMPLE optimisé, sans toutes les sondes, des écritures `temp[i]=/tt[i]=` restent dans 8 blocs de travail (3 avec toutes les sondes) ; SRA ne touche à `temp` dans aucune construction	E1b (fichiers d'analyse `sra`, `optimized`)
[I]	Les 30 (A2) ou 36 (officiel) écritures supplémentaires de l'assembleur sont ces écritures	décompte et forme des tableaux
[H]	Une passe du compilateur qui déplace les écritures (`lim`) décide de les sortir de la boucle ou non ; elle les sort seulement quand toutes les sondes sont actives	fichiers d'analyse non produits
[H]	Les sondes changent la structure du code, et donc la décision de GCC	cohérent avec E1 et E1b, non démontré

Contributions effectives

Ce mémoire a étudié, dans MAYO-C, la façon dont les noyaux AVX2 `P1_times_Vt` et `P1t_times_Vt` traitent la fin des m-vecteurs. Les contributions établies sont les suivantes.

1. Un **comptage rigoureux** dans un modèle précis : le minimum est $\lceil m/32 \rceil$ instructions `vpshufb`, le schéma officiel en utilise $2\lceil m/64 \rceil$, et un gain n'est possible que si $0 < m \bmod 64 \leq 32$. MAYO-1 et MAYO-5 en profitent, pas MAYO-2 ni MAYO-3.

2. Une **mise en œuvre** (A2) pour une fin de vecteur d'au plus 16 éléments, avec une variante SAFE qui ne lit rien au-delà de l'entrée de P_1. Les propriétés de lecture, d'écriture, de tables et de bourrage sont justifiées par le code et vérifiées par des tests.

3. Des **gains mesurés** sur une machine physique, avec une unité statistique précisée : 12 à 29 % sur les noyaux, 10,3 % sur `mayo_sign` pour MAYO-1 et 1,3 % pour MAYO-5, retrouvés lors de deux démarrages de la machine.

4. La **localisation** de la perte de gain de MAYO-5 dans `m_calculate`, quand on mesure la signature complète. On observe aussi, dans le code produit par GCC 16.2.0, des écritures mémoire en plus et inégales entre les deux versions, qui correspondent à cette perte. L'analyse E1b montre que ce sont des stockages de tableaux d'accumulateurs refaits à chaque tour de boucle, et écarte la passe SRA comme cause.

5. Un **dépôt reproductible** (scripts, données brutes, empreintes) avec lequel les neuf analyses donnent exactement les mêmes résultats.

Limites

Le gain sur la signature de MAYO-5 est faible et dépend du choix du compilateur. Le lien de cause à effet entre les écritures en plus et la perte de gain n'est pas démontré, et le modèle chiffré exagère l'effet d'environ 40 %. Les mesures viennent d'une seule machine et d'un seul compilateur, en ticks TSC. Le temps d'exécution indépendant du secret n'est appuyé que par des indices, pas par une preuve. La variante pour une

fin de vecteur de 17 à 32 éléments n'est pas réalisée. Nous ne prétendons à aucune nouveauté absolue.

Questions ouvertes et perspectives

Q1 – Le choix du compilateur. Comprendre pourquoi GCC 16.2.0 laisse ces écritures dans la boucle quand toutes les sondes ne sont pas actives, et les sort de la boucle quand elles le sont. L'analyse E1b a écarté SRA, DSE, PRE et IPA-SRA comme causes uniques (en les désactivant une à une, le doublon reste) et ne permet pas d'accuser FRE (le code change aussi ailleurs). La suite naturelle, toujours sans exécuter le programme, est de produire les fichiers d'analyse des passes qui déplacent les stockages et de PRE (`-fdump-tree-lim2-details`, `-fdump-tree-pre-details`). Ce n'a pas encore été fait : cela attend l'accord de l'encadrant.

Q2 – La cause. Si l'on trouve un moyen de supprimer le doublon sans toucher à autre chose dans le code, comparer alors les temps avec et sans doublon, sur le même code. Les expériences E2 à E5 prévues au départ restent en attente de la décision de l'encadrant.

Q3 – La solidité des résultats. Refaire l'analyse avec Clang et d'autres versions de GCC, et mesurer sur une deuxième machine, si possible avec les compteurs du processeur et en cycles d'horloge (hypothèse de performance H-P).

Q4 – L'extension. Réaliser et valider la variante pour $16 < r \leq 32$; examiner d'autres paramètres que ceux étudiés ici.

Q5 – La garantie. Vérifier, de façon formelle ou avec un outil, que le programme compilé s'exécute en temps indépendant du secret.

Q6 – L'intégration. Décider, en fonction des réponses précédentes et des critères de l'encadrant.

Références

[1] P. W. Shor, *Algorithms for quantum computation : discrete logarithms and factoring*, FOCS 1994.

[2] J. Patarin, *The oil and vinegar signature scheme*, Dagstuhl Workshop on Cryptography, 1997.

[3] A. Kipnis, A. Shamir, *Cryptanalysis of the Oil and Vinegar Signature Scheme*, CRYPTO 1998.

[4] A. Kipnis, J. Patarin, L. Goubin, *Unbalanced Oil and Vinegar Signature Schemes*, EUROCRYPT 1999.

[5] W. Beullens, *Breaking Rainbow Takes a Weekend on a Laptop*, CRYPTO 2022.

[6] W. Beullens, *MAYO : Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*, IACR ePrint 2021/1144 (SAC 2021).

[7] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer, *Nibbling MAYO : Optimized Implementations for AVX2 and Cortex-M4*, IACR ePrint 2023/1683 (version consultée).

[8] NIST, *FIPS 203, 204, 205 : normes de chiffrement et de signature post-quantiques*, août 2024.

[9] NIST, *Call for Additional Digital Signature Schemes for the Post-Quantum Cryptography Standardization Process*, 2022.

[10] P. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO 1996.

[11] D. J. Bernstein, *Cache-timing attacks on AES*, 2005.

[12] V. Arlazarov, E. Dinic, M. Kronrod, I. Faradzev, *On economical construction of the transitive closure of an oriented graph*, Soviet Math. Doklady 11, 1970.

[13] M. R. Albrecht, G. V. Bard, C. Pernet, *Efficient Dense Gaussian Elimination over the Finite Field with Two Elements*, arXiv :1111.6549, 2011.

[14] W. Beullens, M.-S. Chen, S.-H. Hung, M. J. Kannwischer, B.-Y. Peng, C.-J. Shih, B.-Y. Yang, *Oil and Vinegar : modern parameters and implementations*, IACR TCHES 2023(3).

[15] Dépôt officiel `PQCMayo/MAYO-C`, commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`.

Annexe A

Inventaire des archives et reproduction

État au 22 octobre 2026 (dernier inventaire vérifié) ; les ajouts postérieurs (E1b, identification des noyaux) sont dans le dépôt `mayo-a2-thesis`. Toutes les empreintes sont des SHA-256.

A.1 Versions définitives à citer

Fichier	SHA-256	Contenu
`A2_partie1.tar.gz`	`cbc0cc297ee0a0790b3cb7513d0384c6c323cfe26b6180b93fc7bd1b115477a6`	notes, code, scripts, analyses, brut S2
`A2_partie2.tar.gz`	`d8dbf1b169f249c720a6e74eb0b9c6e6c9fe44eae6087ef6fe188aa21ab0fcab`	brut PH1 et CT1
`A2_partie3.tar.gz`	`00d12e885de273e7851b270f8b53b780eb52fd5bbbe969b1dfe73402b687ac33`	brut S1
patch `0001`	`e6c03362cbf974cf750f340453de874d667b5670a7e60eddafa54b35e29e0cea`	prototype A2
patch `0002`	`751ec21fbd059e51a4b54f193f3e346a889d4b361cfdf78a3ab77c3e891b6280`	sondes
patch `0003`	`b590a81d479fd4a32883a60efe4bb5525fb9f45bf92a1acd70f60b265596112d`	contrôle des sondes
patch `0004`	`9c460d0de2193805f3b0c1716f62bf1390ca543778ccfb0022d995ef254a10c6`	analyse statique E1
`MAYO-C-A2_source_ e72d407.tar.gz`	`e2252a0b51ab827bd097e5a0df73dcbb7ca13539af82b1e728737c58800f6833`	arbre S1/S2
`MAYO-C-A2-phases-v2_source.tar.gz`	`ecb0941b086affb7189151af4b58355acb39edd4de8d7089b34ac7328cf19ef3`	arbre PH1/CT1
`E1_kit.tar.gz`	`efe0b71ca5dc1553b0c0a841dc59096216a214080d988ba32731cbe2fcde18f3`	`run_E1.sh`, `e1_analyze.py`

Archive E1b (sorties de compilation, dumps GCC, désassemblages) : `e1b_gcc16.tar.gz`, SHA-256 `49452120b1b787ad2aed0f30cbcece4f7584a6063768757d92214e264dcd3faa`.

Archives brutes internes : S1 `ebc01dfe`…, S2 `b1a69769`…c723, PH1 `e1a88aec`…98cc, CT1 `0506c2b5`…92de. Les archives de la phase VM (`A2_donnees_et_journaux`,

`tail_aware_etude`, `archive_piste_batch_EF`, etc.) sont de l'*historique* et ne doivent jamais être présentées comme performances matérielles.

A.2 Identification exacte du code

Le code est déterminé par l'arbre git (indépendant de l'horodatage des commits). Base : `PQCMayo/MAYO-C` commit `74de637a7c4a7f9fafa888183bc05f5cf920cd2d`, puis `git am` des patchs dans l'ordre. Arbres attendus : après `0001` `87d1729c`…(S1, S2) ; après `0002` `c8c37c97`… ; après `0003` `92bcede1`…(PH1, CT1) ; après `0004` `bf6955da`….

```
git clone https://github.com/PQCMayo/MAYO-C && cd MAYO-C
git checkout 74de637a7c4a7f9fafa888183bc05f5cf920cd2d
git am 0001-*.patch 0002-*.patch 0003-*.patch 0004-*.patch
git rev-parse HEAD^{tree}   # attendu : bf6955dae657abb087b58e13e314b946e0d283fe
```

A.3 Reproduire les analyses

Prérequis : Linux, `python3` avec `numpy`, `tar`, `sha256sum`. Les trois archives dans un dossier D :

```
sh reproduire_analyses.sh D travail_neuf
```

Le script vérifie les empreintes, relance les neuf analyses et affiche `OK` si la sortie est identique (résultat obtenu : 9 sur 9). Les sorties couvrent la synthèse S1/S2 (`synthese_sessions.py`), la comparaison (`compare_sessions.py`), la décomposition (`phases_analyze.py`), le contraste avec/sans sondes (`phases_vs_ref.py`) et le contrôle CT1 (`controle_analyze.py`).

A.4 Refaire les mesures

Conditions : Linux, AVX2, GCC $\geq$ 13 (mesures : 16.2.0), cmake, `python3+numpy`, gouverneur `performance` (`sudo cpupower frequency-set -g performance`), cœur 1 dédié, aucun autre travail pendant l'exécution.

Étape	Commande (racine de l'arbre correspondant)
S1, S2	`SESSION=<étiquette> sh run_session_independante.sh`
PH1	`SESSION=<étiquette> sh test/run_phases.sh`
CT1	`SESSION=<étiquette> sh test/run_controle.sh`
E1	`SESSION=<étiquette> sh test/run_E1.sh`
E1b	`python3 -I e1b.py <dossier_E1> <dossier_neuf>`

Chaque `machine.txt` enregistre date UTC, noyau, compilateur, microcode, gouverneur, `boot_id` et alimentation. La reproduction des *mesures* donnera des valeurs voisines et non identiques.

Annexe B

Identification des noyaux dans le code généré (GCC 16.2.0, MAYO-5)

Identification par le pas d'adresse sur P_1 (section 2.3). « Écritures » : écritures de 32 octets dans le corps de boucle.

Tab. B.1 : Instructions / écritures de 32 octets des boucles de travail selon la construction.

Construction	`P1` off.	`P1` A2	`P1t` off.	`P1t` A2
`ctx_official`	123 / 29	–	162 / 65	–
`ctx_ab`, `none`, `esk`, `mcalc`	123 / 29	135 / 52	162 / 65	139 / 52
`ctx_phases` (toutes sondes)	123 / 29	105 / 22	126 / 29	109 / 22

Les différences « sans toutes les sondes » moins « toutes sondes » sont : A2 `P1` $+30$ ($52-22$), A2 `P1t` $+30$, officiel `P1t` $+36$ ($65-29$), officiel `P1` 0.

Annexe C

Exemple jouet complet de MAYO sur $\mathbb{F}_{16}$

Cet exemple, calculé à la main et vérifié par un programme, illustre la section 1.1.1. Les éléments de $\mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4 + x + 1)$ sont écrits en hexadécimal (A $= 10$, B $= 11$, ..., F $= 15$) ; l'addition est le OU exclusif.

Paramètres

$n = 3$, $v = 2$, $o = 1$, $m = 2$, $k = 2$: on a bien $o < m$ et $k\,o = m$. Le secret est $O = (\mathrm{B}, 8)^{\mathsf{T}}$.

Clé publique (blocs de chaque équation $\ell = 1, 2$)

$$P_1^{(1)} = \begin{pmatrix} 8 & 9 \\ 0 & 6 \end{pmatrix},\ P_2^{(1)} = \begin{pmatrix} 2 \\ 2 \end{pmatrix},\ P_3^{(1)} = (6); \qquad P_1^{(2)} = \begin{pmatrix} 8 & \mathrm{A} \\ 0 & 8 \end{pmatrix},\ P_2^{(2)} = \begin{pmatrix} \mathrm{B} \\ \mathrm{C} \end{pmatrix},\ P_3^{(2)} = (\mathrm{E}).$$

Le bloc $P_3^{(\ell)} = \mathrm{Upper}\big(-O^{\mathsf{T}}(P_1^{(\ell)}O - P_2^{(\ell)})\big)$ est calculé avec le secret.

Signature

Le haché salé est supposé égal à $t = (\mathrm{F}, 9)$. Vinaigres tirés : $V = \begin{pmatrix} 2 & 3 \\ 3 & \mathrm{A} \end{pmatrix}$ (lignes v_1, v_2).

On calcule

$$Y^{(1)} = VP_1^{(1)}V^{\mathsf{T}} = \begin{pmatrix} 8 & 7 \\ 6 & 4 \end{pmatrix}, \qquad Y^{(2)} = VP_1^{(2)}V^{\mathsf{T}} = \begin{pmatrix} 1 & 4 \\ 3 & 9 \end{pmatrix}.$$

Le second membre est $y = t + \big(u_{11} + z\,u_{12} + z^2 u_{22}\big) = (8, 7)$, où z est la classe de x modulo $z^2 + z + 1$ et u_{ij} le m-vecteur des coefficients de $Y^{(\cdot)}$. La matrice du système est $A = \begin{pmatrix} 2 & 1 \\ \mathrm{F} & \mathrm{D} \end{pmatrix}$; son déterminant vaut $6 \neq 0$, donc il y a une solution unique $x = (x_1, x_2) = (8, \mathrm{B})$. La signature est $S = (s_1, s_2)$ avec $s_i = (v_i + Ox_i \,\|\, x_i)$:

$$s_1 = (5, \mathrm{F}, 8), \qquad s_2 = (\mathrm{A}, \mathrm{D}, \mathrm{B}).$$

Vérification

On évalue $P^{(\ell)}(s_1)$, $P^{(\ell)}(s_2)$ et $P'^{(\ell)}(s_1, s_2)$: $P(s_1) = (\mathrm{A}, 5)$, $P(s_2) = (9, \mathrm{B})$, $P'(s_1, s_2) = (\mathrm{D}, 4)$. Alors

$$P^\star(S) = P(s_1) + z\,P'(s_1, s_2) + z^2 P(s_2) = (\mathrm{A}, 5) + (6, 9) + (3, 5) = (\mathrm{F}, 9) = t.$$

La signature est acceptée. Si l'on modifie un seul élément de s_1, l'égalité est perdue et la signature est refusée.